
Computer Organization & Architecture

計算機組織

Chapter 3

Arithmetic for Computers

Prof. Jong-Yih Kuo
Computer Science and Information Engineering
National Taipei University of Technology

3.1 Introduction

- ❑ Operations on integers
 - Addition and subtraction
 - Multiplication and division
 - Dealing with overflow

- ❑ Floating-point real numbers
 - Representation and operations

3.2 Addition and Subtraction

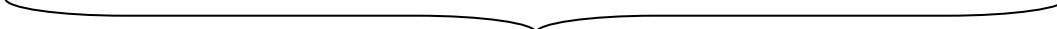
- ❑ Designing MIPS ALU for arithmetic and logic operations
 - add, sub: two's complement adder/subtractor with overflow detection
 - and, or, nor : logical AND, logical OR, logical NOR
 - if Zero
 - overflow exception
 - slt (set on less than): two's complement adder with inverter, check sign bit of result

Defining Number System First

- Binary integers (base 2)

- decimal: $0, \dots, 2^{n-1}$

0000 0001 0010 0011 0100 0101 0110 0111 1000 1001...



n bits

- bit strings are finite

- Some fractions and real numbers, finite bits is not enough
 - so overflow & approximation errors
 - e.g., represent $1/3$ as binary!

Negative Integers

□ Sign Magnitude One's Complement 2's Complement

000 = 0	000 = 0	000 = 0
001 = +1	001 = +1	001 = +1
010 = +2	010 = +2	010 = +2
011 = +3	011 = +3	011 = +3
100 = 0	100 = -3	100 = -4
101 = -1	101 = -2	101 = -3
110 = -2	110 = -1	110 = -2
111 = -3	111 = 0	111 = -1
ambiguous zero	ambiguous zero	unequal number of negatives and positives; unique zero

□ Issues

- balance – equal number of negatives and positives
- ambiguous zero – more than one zero representation, not ease for arithmetic operations

MIPS – 2's Complement

□ 32 bit signed numbers

0000 0000 0000 0000 0000 0000 0000 0000_{two} = 0_{ten}
0000 0000 0000 0000 0000 0000 0000 0001_{two} = + 1_{ten}
0000 0000 0000 0000 0000 0000 0000 0010_{two} = + 2_{ten}

...

0111 1111 1111 1111 1111 1111 1111 1110_{two} = + 2,147,483,646_{ten}

0111 1111 1111 1111 1111 1111 1111 1111_{two} = + 2,147,483,647_{ten} ← **maxint**

1000 0000 0000 0000 0000 0000 0000 0000_{two} = - 2,147,483,648_{ten} ← **minint**

1000 0000 0000 0000 0000 0000 0000 0001_{two} = - 2,147,483,647_{ten}

1000 0000 0000 0000 0000 0000 0000 0010_{two} = - 2,147,483,646_{ten}

...

1111 1111 1111 1111 1111 1111 1111 1101_{two} = - 3_{ten}

1111 1111 1111 1111 1111 1111 1111 1110_{two} = - 2_{ten}

1111 1111 1111 1111 1111 1111 1111 1111_{two} = - 1_{ten}

Negative integers are exactly those that have leftmost bit 1

2's Complement Operations

- ❑ Negation: To negate complement
 - invert all bits and add 1
- ❑ Sign Extension (e.g. 4bit -> 8 bit)
 - convert an n-bit integer into an integer with more than n bits
 - to make a narrow integer fill a wider word
 - replicate the **most significant bit (msb)** of the original number to fill the new bits to its left

4-bit		8-bit
0010	= XXXX	0010 = 0000 0
1010	= XXXX	1010 = 1111 1

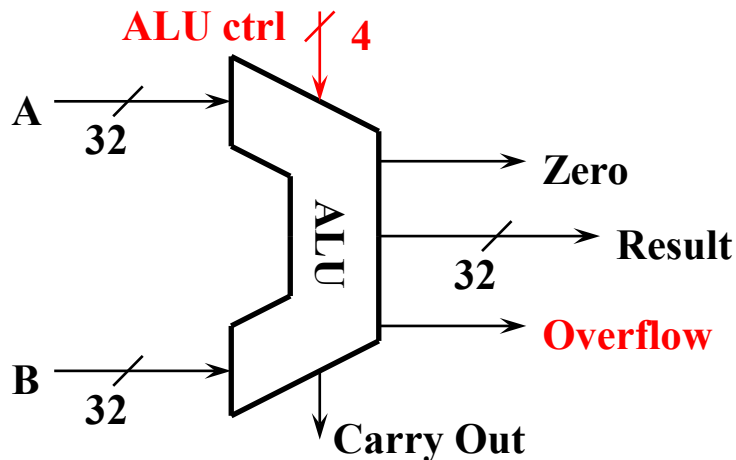
Functional Specification of ALU

□ ALU (Arithmetic Logic Unit)

- **Unsigned: Carry out**
- **Singed: Overflow**

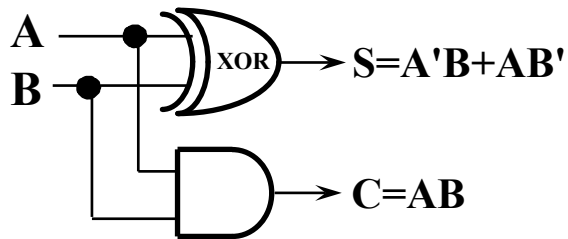
ALUctrl	Function
0000	and
0001	Or
0010	add
0110	subtract
0111	set-less-than
<u>1100</u>	nor

Ainvert Bnegate Operation (MUX_{Selector})



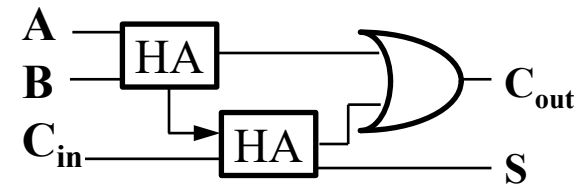
Adder

□ Adder Logic Unit

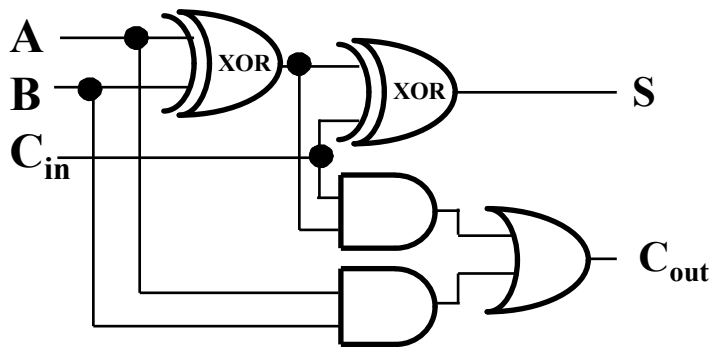


Half Adder

A	B	S	C _{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

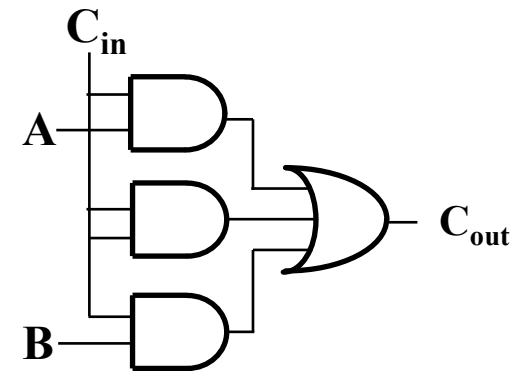


Full Adder implemented by two Half Adders



Full Adder

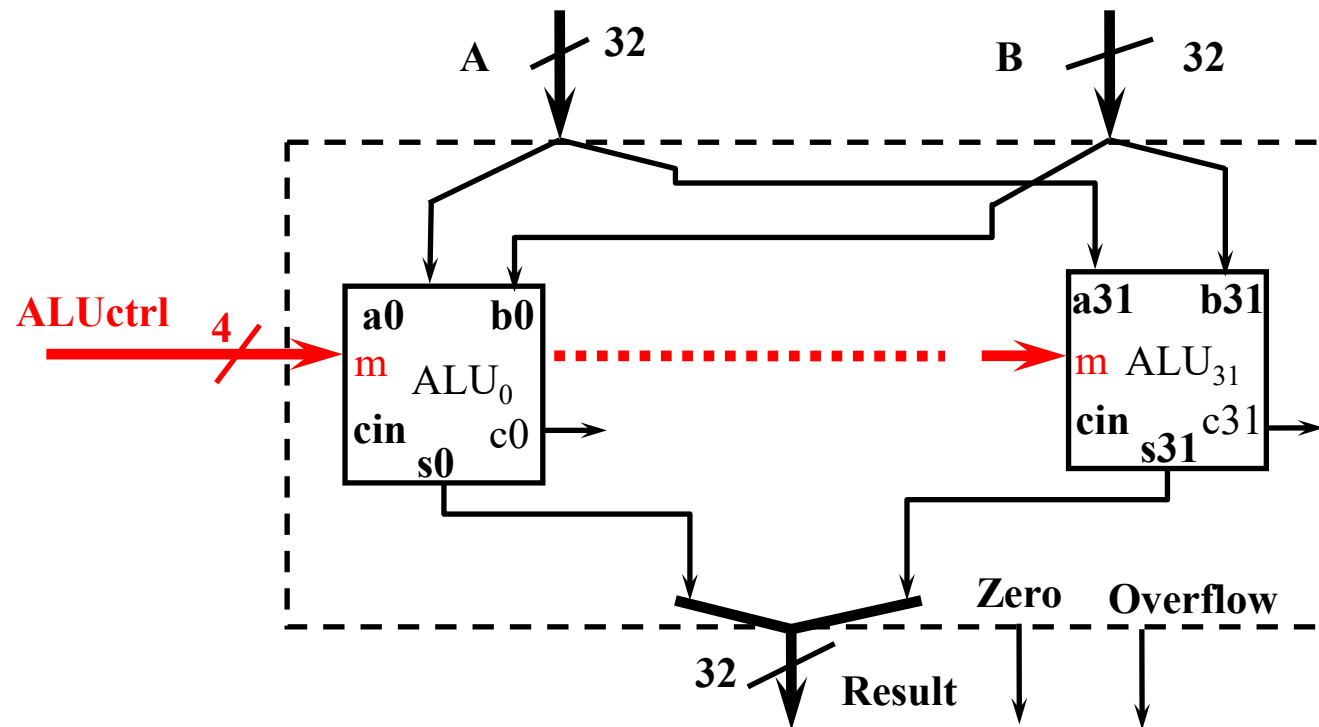
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Carry out

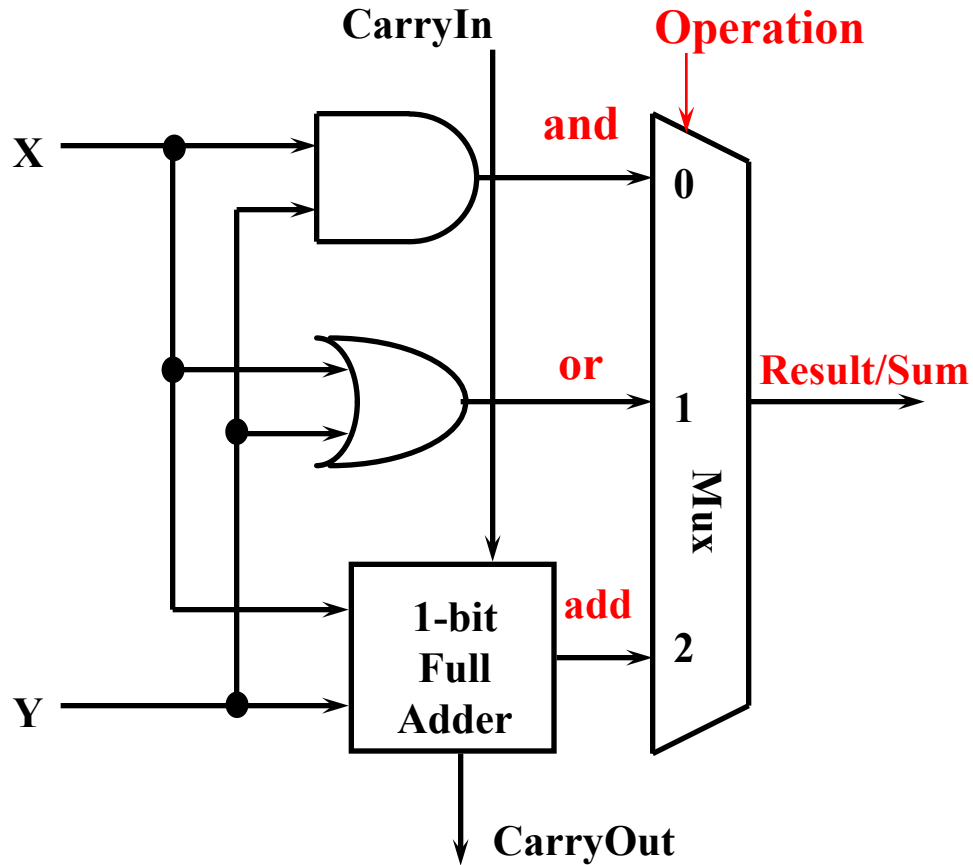
A Bit-slice ALU

- Design methods: Divide and Conquer (切割問題到更簡化，並予以克服)
 - solve part of the problem and glue together the solution
 - **s0~s31: sum, c0~c21: carry out, m: mux**

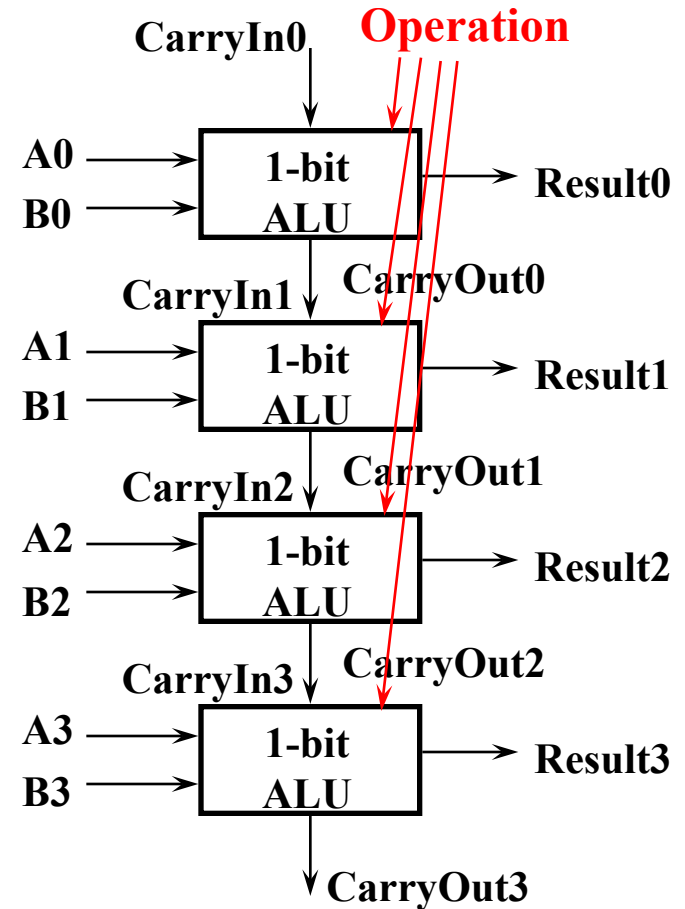


A 4-bit ALU

1-bit ALU



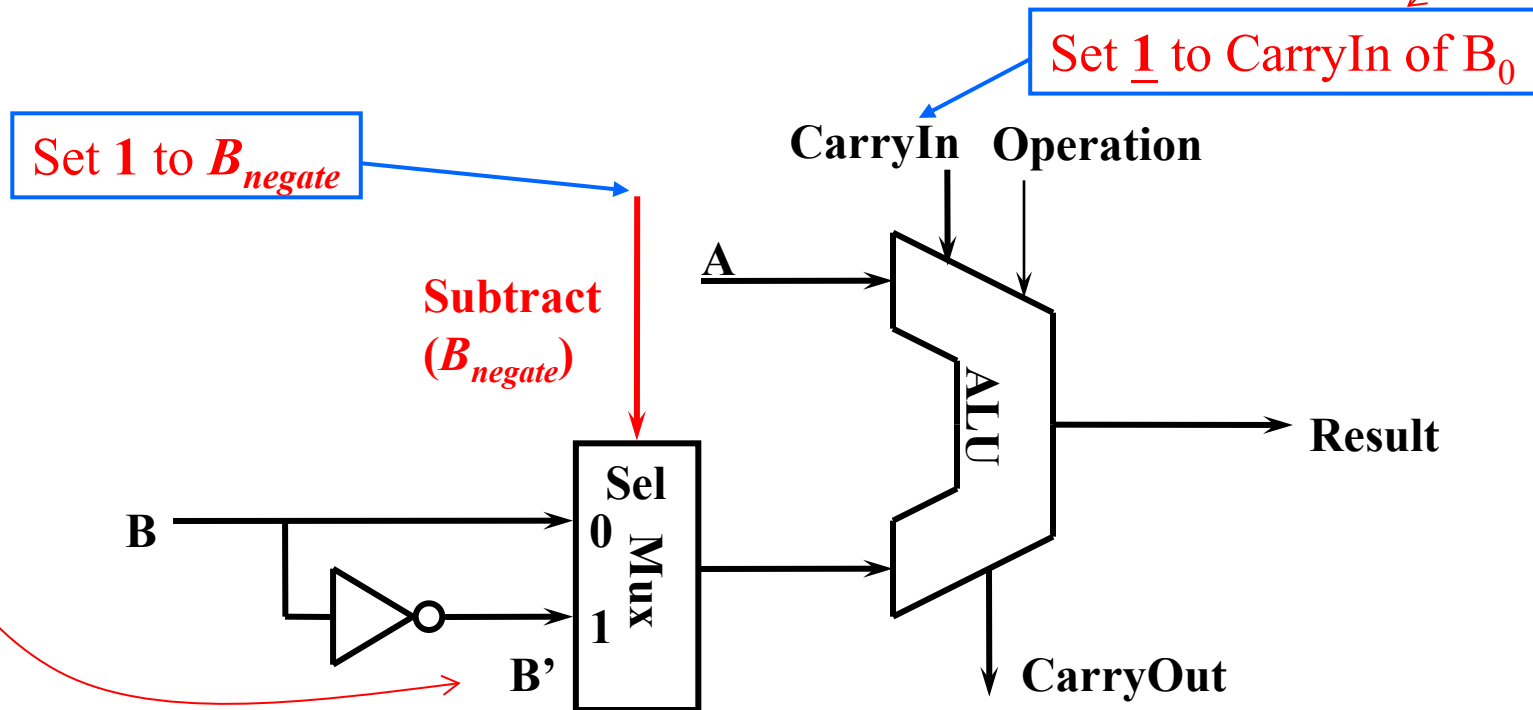
4-bit ALU



Subtraction

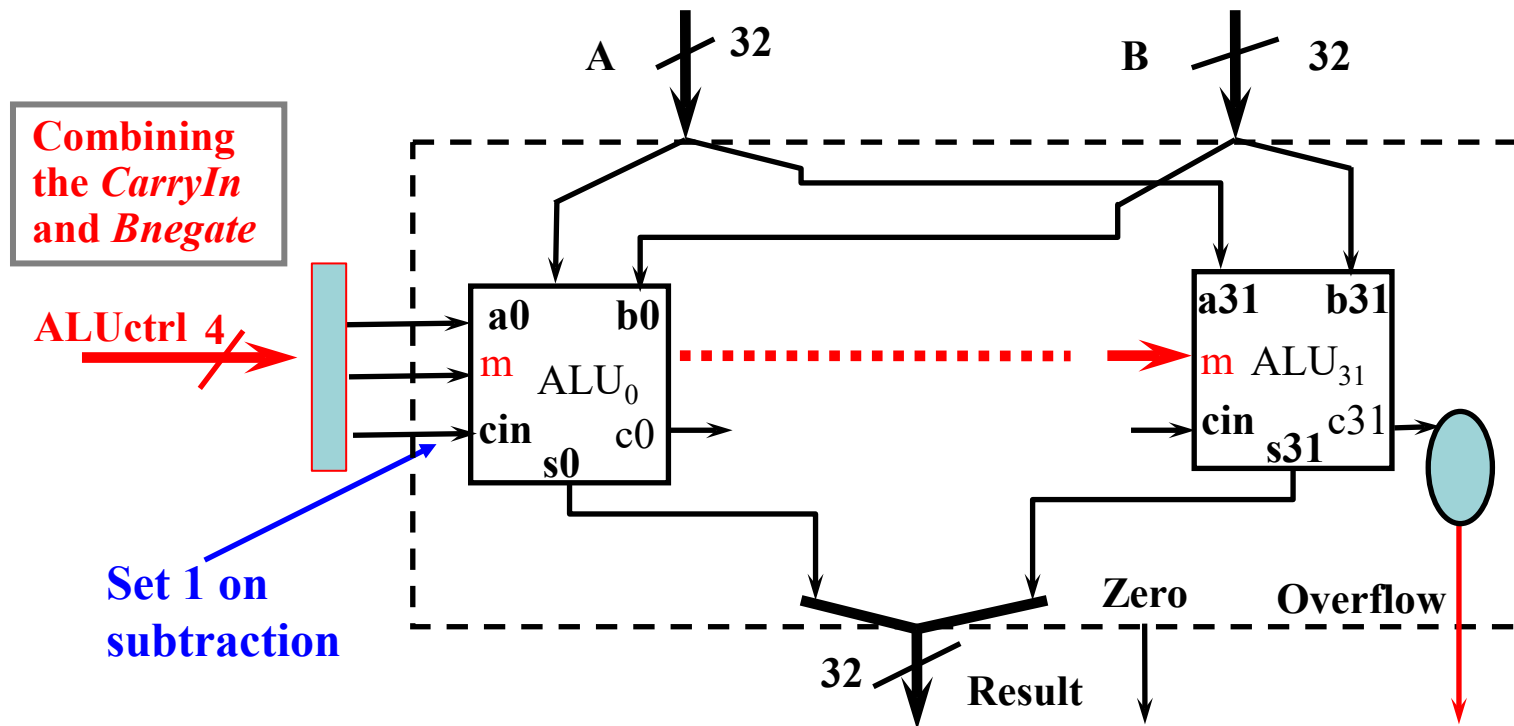
□ 2's complement (e.g. $+5 \rightarrow -5$)

- 每一個 bit inverse
- 在 B_0 地方 +1 (cin of first stage)
- $A - B = A + (-B) = A + (B' + 1) = A + B' + 1$



Revised Diagram

- LSB and MSB need to do a little extra (後面有設計圖)
 - **MSB** (most significant bit) 最高有效位元, **LSB** (least significant bit) 最低有效位元



Dealing with Overflow

- ❑ Some languages (e.g., C) ignore overflow
 - Use MIPS addu, addui, subu instructions
- ❑ Other languages (e.g., Ada, Fortran) require raising an exception
 - Use MIPS add, addi, sub instructions
 - On overflow, invoke exception handler
 - Save PC in exception program counter (EPC) register
 - Jump to predefined handler address (coprocessor)
 - mfc0 (move from coprocessor register) instruction can retrieve EPC value, to return after corrective action

Integer Addition

□ Example: no overflow

○ $7 + 6 = 13$

$$\begin{array}{r}
 +7: \quad 0000 \ 0000 \ \dots \ 0000 \ 0111 \\
 +6: \quad 0000 \ 0000 \ \dots \ 0000 \ 0110 \\
 \hline
 +13: \quad 0000 \ 0000 \ \dots \ 0000 \ 1101
 \end{array}$$

Decimal	Binary	Decimal	2's complement
0	0000	0	0000
1	0001	-1	1111
2	0010	-2	1110
3	0011	-3	1101
4	0100	-4	1100
5	0101	-5	1011
6	0110	-6	1010
7	0111	-7	1001
		-8	1000

□ Overflow if result out of range (結果超出範圍)

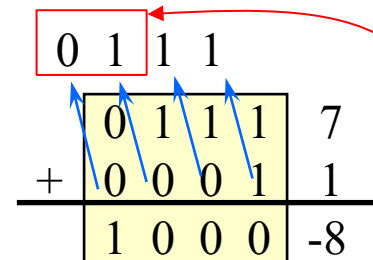
○ Adding +ve and -ve operands, no overflow

○ Adding two +ve operands (2正數相加)

➤ Overflow if result sign is **1**

○ Adding two -ve operands (2負數相加)

➤ Overflow if result sign is **0**



Carry into MSB $\neq$ Carry out of MSB (偵測overflow)

Integer Subtraction

□ Add negation of second operand

○ Example: $7 - 6 = 7 + (-6)$

+7:	0000	0000	...	0000	0111
-6:	1111	1111	...	1111	1010
+1:	0000	0000	...	0000	0001

□ Overflow if result out of range (結果超出範圍)

○ Subtracting two +ve or two -ve operands, no overflow

○ Subtracting +ve from -ve operand $((-\text{ve}) - (\text{ve}))$ (負數減正數)

➤ Overflow if result sign is 0

○ Subtracting -ve from +ve operand $((\text{ve}) - (-\text{ve}))$ (正數減負數)

➤ Overflow if result sign is 1

	1	0	0	0	
	1	1	0	0	-4
+	1	0	1	1	-5
	0	1	1	1	7

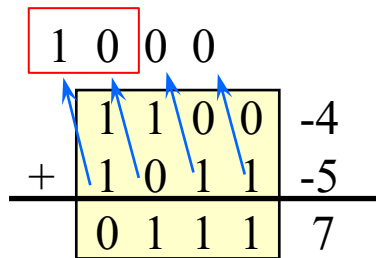
Carry into MSB $\neq$ Carry out of MSB

Overflow Detection

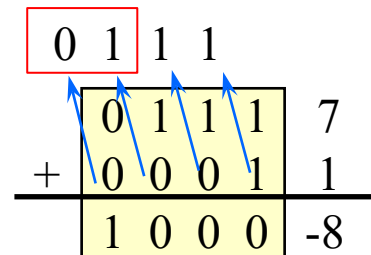
❑ Overflow: result too big/small to represent

- When adding operands with different signs, overflow cannot occur!
- Overflow occurs when adding
 - 2 positive numbers and the sum is negative
 - 2 negative numbers and the sum is positive
 - => sign bit is set with the value of the result
- Legal range: $-8 \leq 4\text{-bit binary number} \leq 7$

$-4 - 5 = -9$



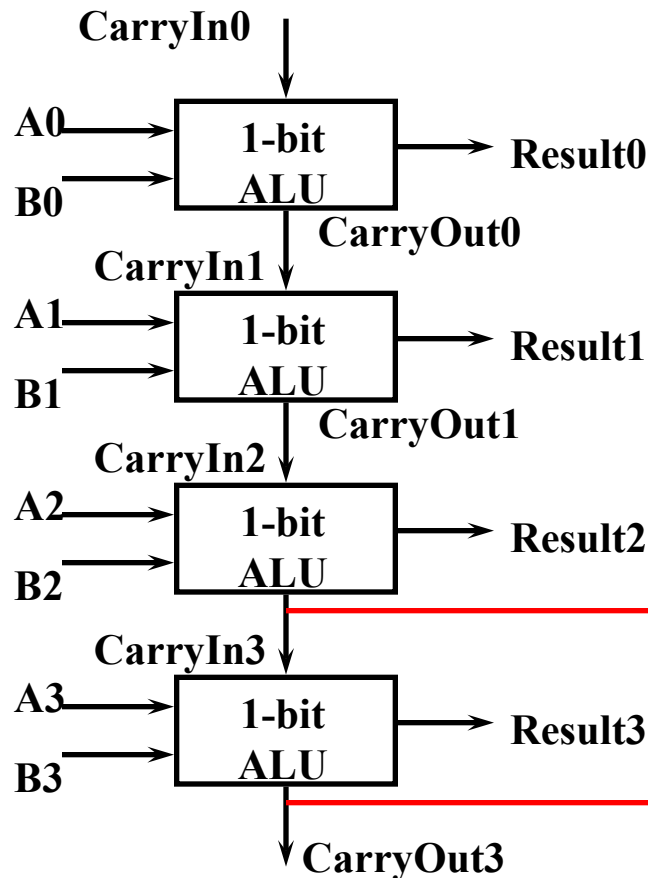
$7 + 1 = 8$ (超出範圍)



- Overflow if: **Carry into MSB $\neq$ Carry out of MSB**

Overflow Detection Logic

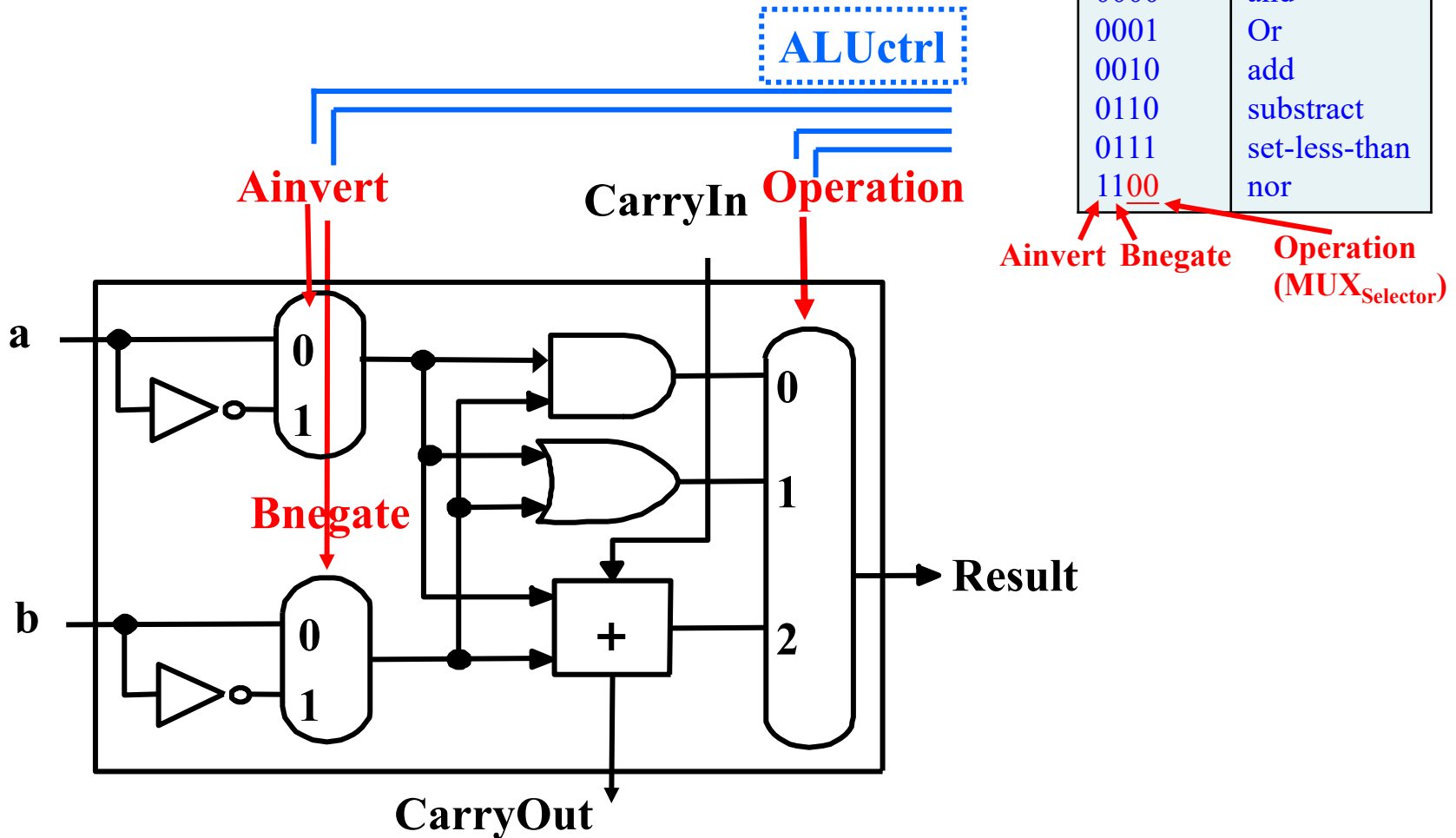
❑ $\text{Overflow} = \text{CarryIn}[N-1] \text{ XOR } \text{CarryOut}[N-1]$



CarryIn[N-1]	CarryOut[N-1]	XOR
0	0	0
0	1	1
1	0	1
1	1	0

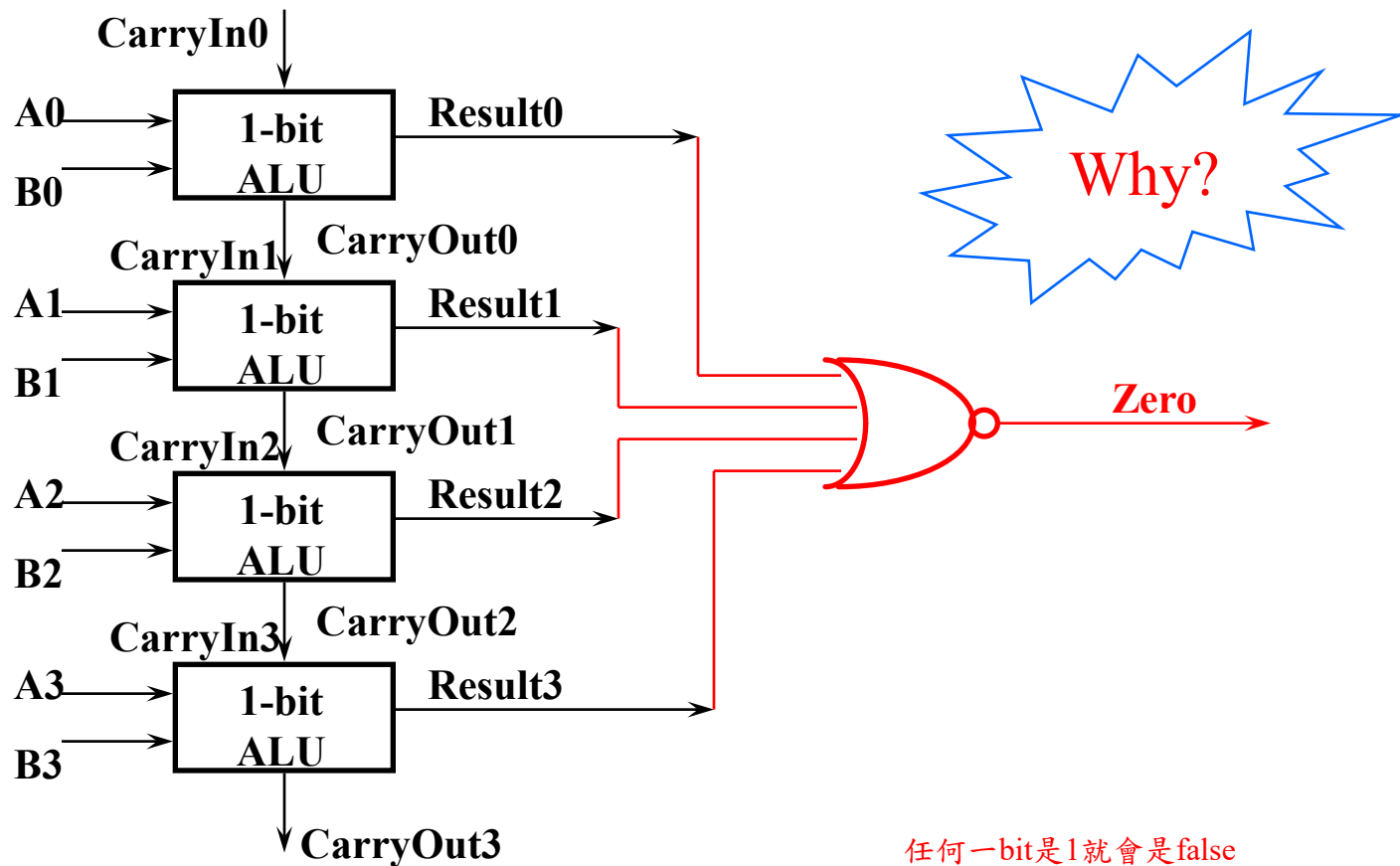
Nor Operation

□ $A \text{ nor } B = (\text{not } A) \text{ and } (\text{not } B)$



Zero Detection Logic

- Zero Detection Logic is a one BIG NOR gate (support conditional jump, beq, bne)

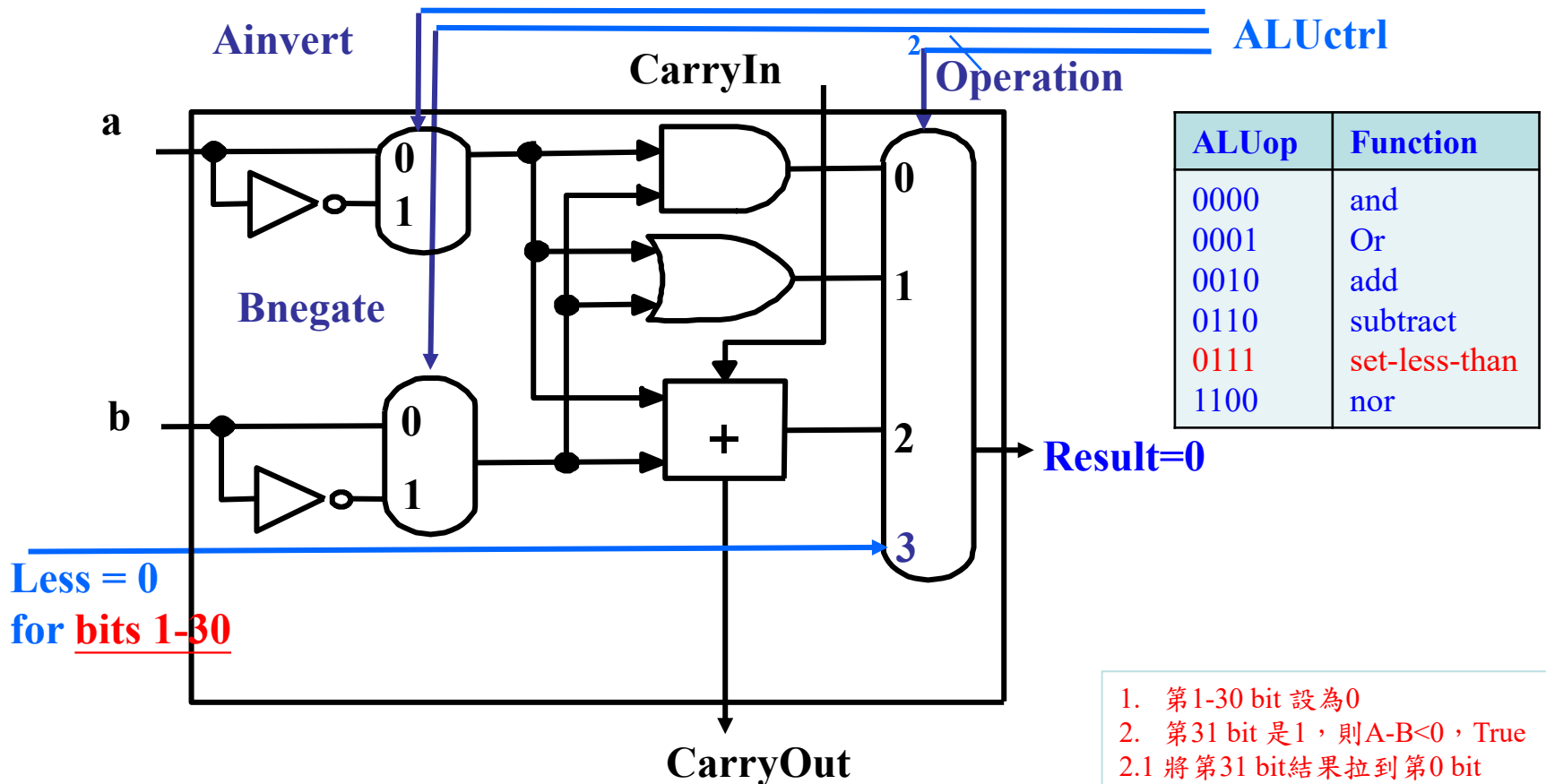


Set on Less Than

□ if $(A-B) < 0$ then Result = 1 (00...01)

0000...000000...01

○ Result = 0 for **bits 1-30** (無論結果是0或1，1-30 bit 都是0)



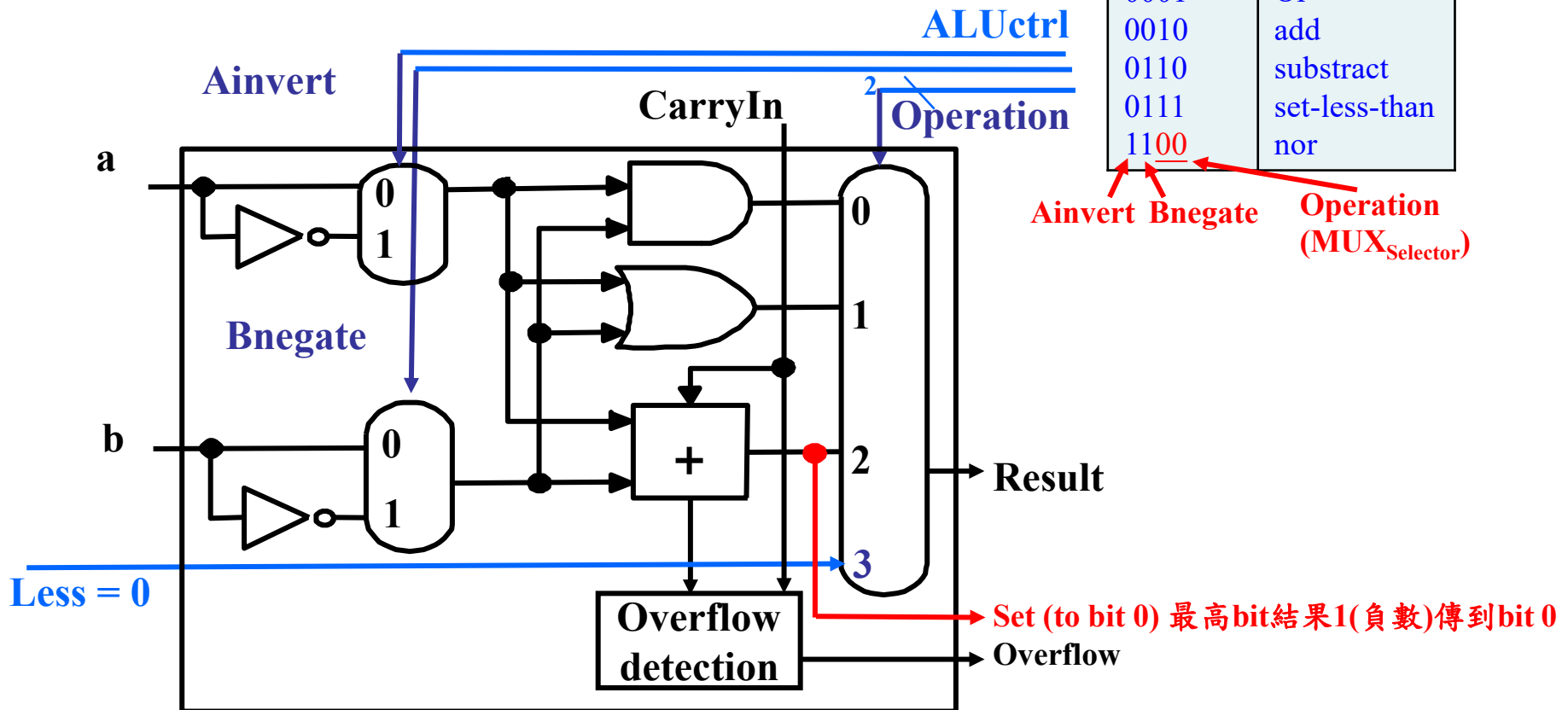
1. 第1-30 bit 設為0
2. 第31 bit 是1，則 $A-B < 0$ ，True
- 2.1 將第31 bit 結果拉到第0 bit
- 2.2 再將第31 bit 設為 0

Set on Less Than

○ Sign bit (Bit 31) in ALU

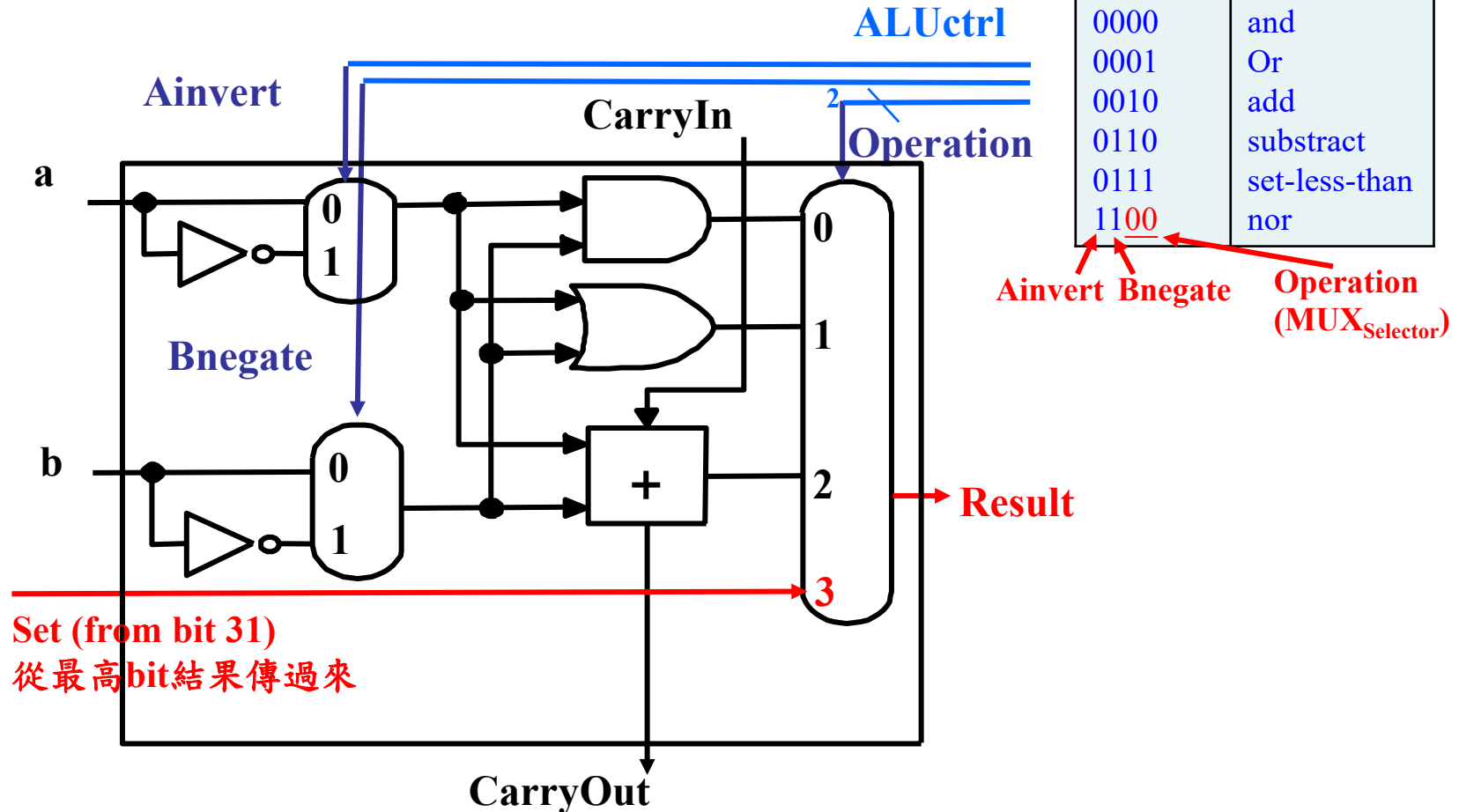
➤ A, B 相減結果最高bit為1表示負數，即 $A-B < 0$

ALUop	Function
0000	and
0001	Or
0010	add
0110	subtract
0111	set-less-than
1100	nor

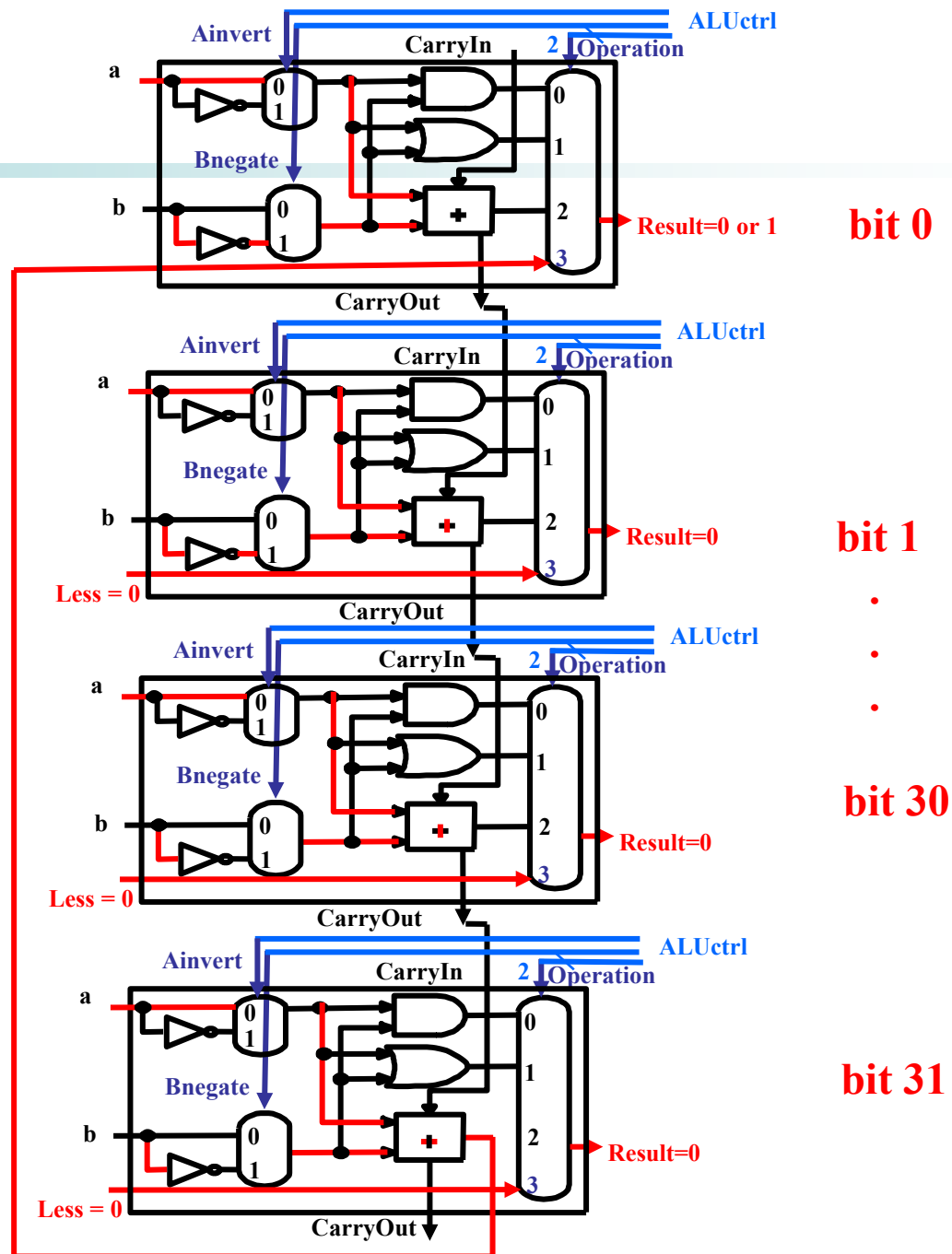


Set on Less Than

- Bit 0 in ALU

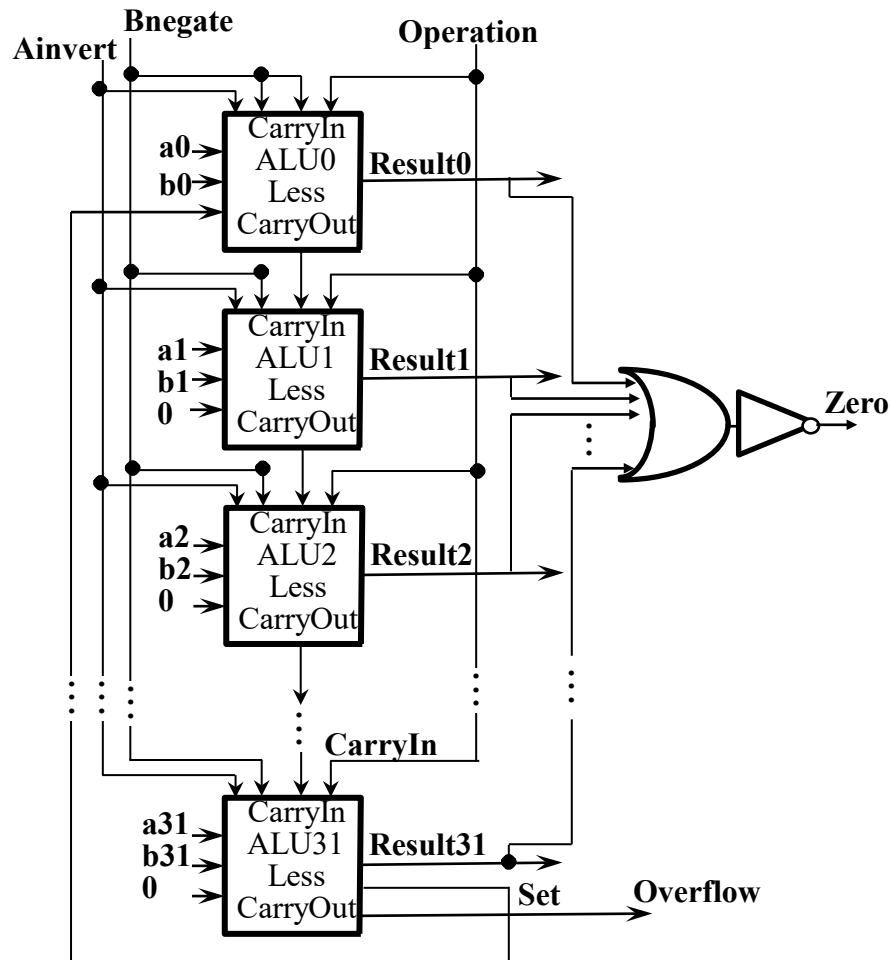


Set on Less Than



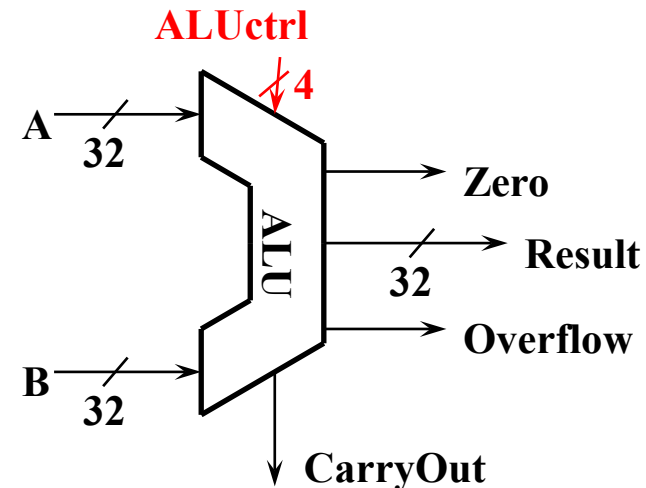
二的補數，
A-B的 bit 31加法結果為1，
代表負數，拉到bit 0結果為1，
其他位數為0。

An ALU with Ripple Carry Adder



ALUop	Function
0000	and
0001	Or
0010	add
0110	subtract
0111	set-less-than
1100	nor

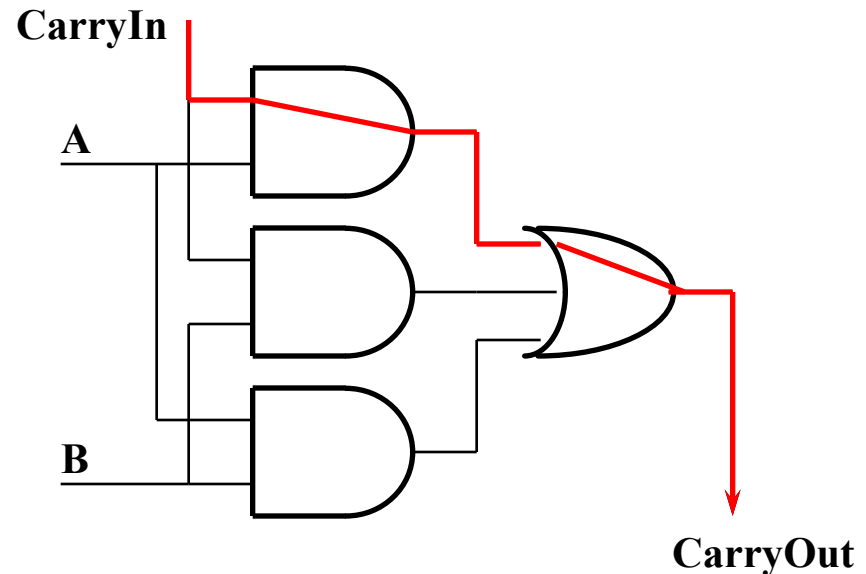
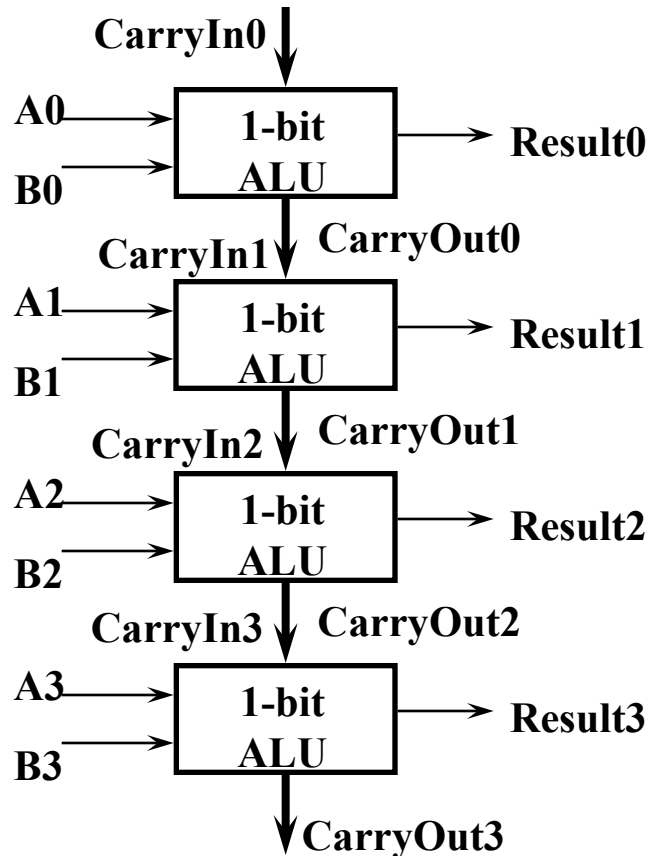
Ainvert **Bnegate** **Operation (MUX_{Selector})**



Ripple 波紋、漣漪

Problems with Ripple Carry Adder

- ❑ Carry bit may have to propagate from LSB to MSB
 - worst case delay: N-stage delay (32-bit delay 64 gates)
 - Design Trick: look for **parallelism** and throw hardware at it



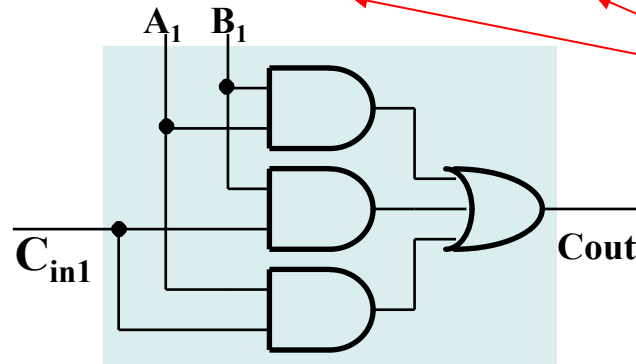
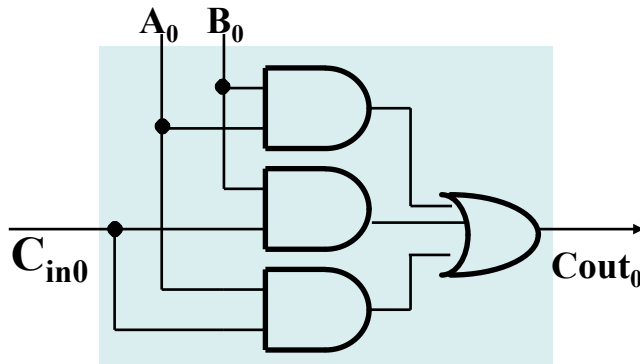
CarryIn to CarryOut has a 2-gate delay

Carry Lookahead: Theory (I)

□ Carry Out = (B * Carry In) + (A * Carry In) + (A * B)

$$Cin1 = Cout0 = (A0 * B0) + (A0 * Cin0) + (B0 * Cin0) = (\textcolor{red}{A0 * B0}) + (\textcolor{blue}{A0 + B0}) * Cin0$$

$$Cin2 = Cout1 = (A1 * B1) + (A1 * Cin1) + (B1 * Cin1) = (\textcolor{red}{A1 * B1}) + (\textcolor{blue}{A1 + B1}) * Cin1$$



有規則

□ Substituting Cin1 into Cin2:

$$Cin2 = (\textcolor{red}{A1 * B1}) + (A1 * A0 * B0) + (A1 * A0 * Cin0) + (A1 * B0 * Cin0) + (B1 * A0 * B0) + (B1 * A0 * Cin0) + (B1 * B0 * Cin0)$$

○ Cin2 **ONLY** depends on **Cin0, A0, B0, A1, and B1**

➤ Cin2 **NOT** depend on the **carry coming out of bit 0 (Cout0)**

➤ not to wait for the carry to propagate through the lower bits

(可以跳過延遲時間)

Carry Lookahead: Theory (II)

□ Define two new terms

Generate Carry at Bit i: $g_i = A_i * B_i$

Propagate Carry via Bit i: $p_i = A_i + B_i$

← A B 都要存在

← A B 任一個存在

$$C_{in1} = (A_0 * B_0) + (A_0 + B_0) * C_{in0} = g_0 + p_0 * C_{in0}$$

$$C_{in2} = (A_1 * B_1) + (A_1 + B_1) * C_{in1} = g_1 + p_1 * (g_0 + p_0 * C_{in0}) = g_1 + p_1 * g_0 + p_1 * p_0 * C_{in0}$$

$$C_{in3} = (A_2 * B_2) + (A_2 + B_2) * C_{in2} = g_2 + p_2 * (g_1 + p_1 * g_0 + p_1 * p_0 * C_{in0})$$
$$= \underline{g_2 + p_2 * g_1 + p_2 * p_1 * g_0 + p_2 * p_1 * p_0 * C_{in0}}$$

← 目前 A B 都要存在 or

← A B 任一個存在 and 前一個有進位 or

....

□ Carry going into bit 3 is 1 if, generate

a carry at bit 2 (g_2), or

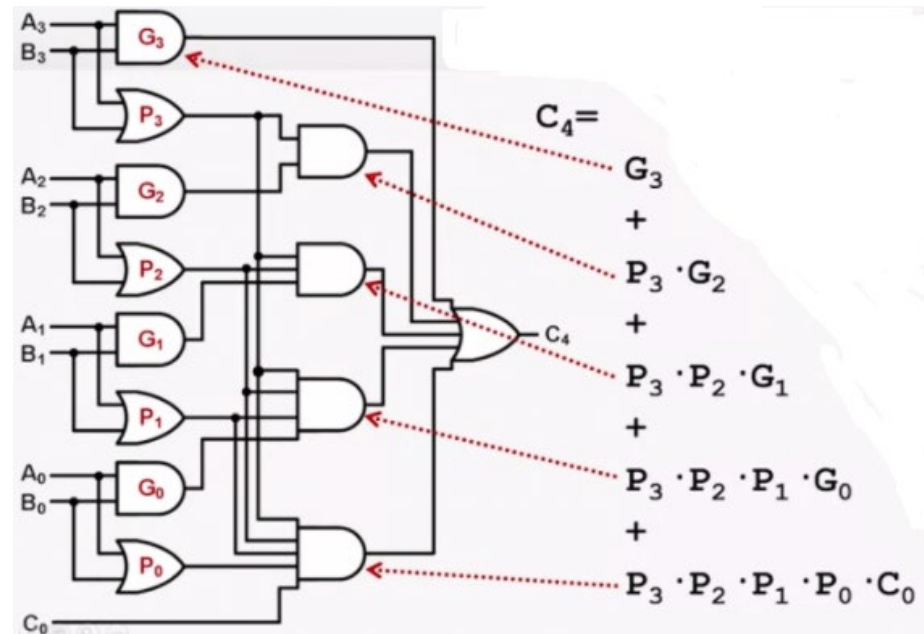
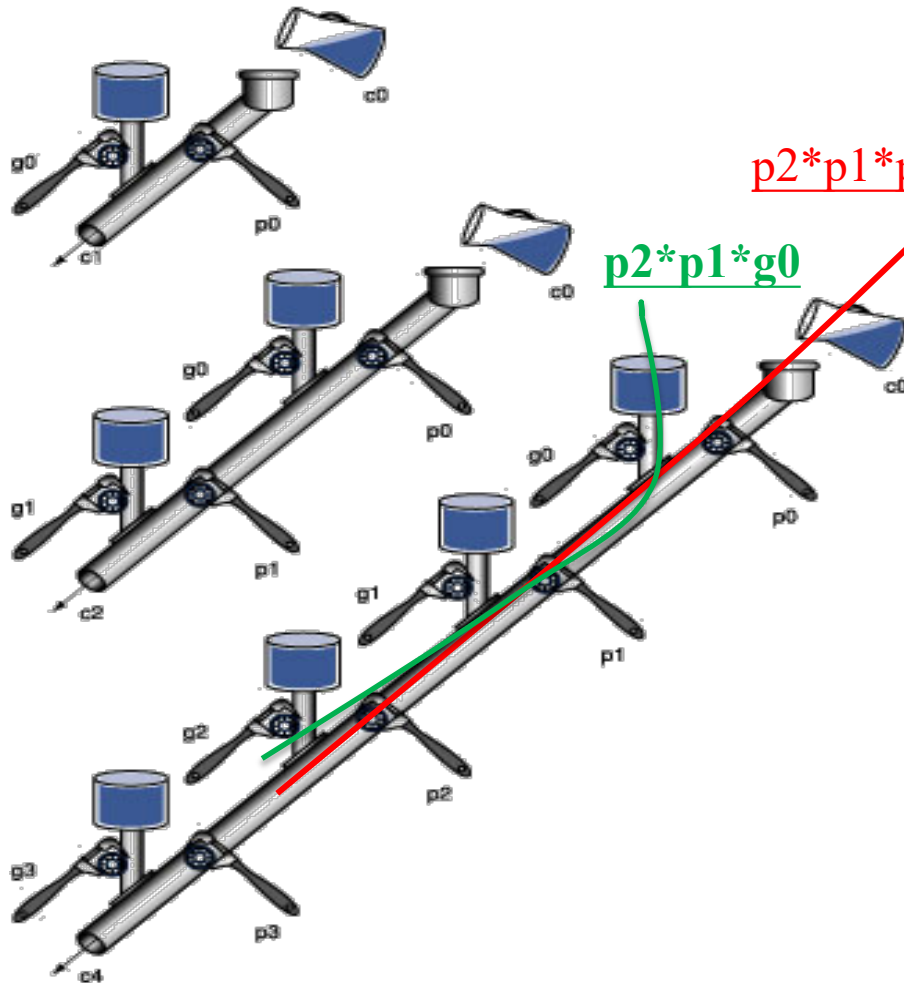
a carry at bit 1 (g_1) and bit 2 allows it to propagate ($p_2 * g_1$), or

a carry at bit 0 (g_0) and bit 1 and bit 2 allows it to propagate, or ($p_2 * p_1 * g_0$), Or

the carry in at bit 0 (C_{in0}) and bit 0, bit 1, and bit 2 allows it to propagate ($p_2 * p_1 * p_0 * C_{in0}$)

Carry Lookahead: Theory (III)

- A Plumbing Analogy for Carry Lookahead for 1st, 2nd, and 4th carry-in's



Gate Delays for Lookahead Carry

- 1 delays

- $g_i = A_i * B_i$

- $p_i = A_i + B_i$

- Total 3 delays

- $C_{in1} = g_0 + p_0 * C_{in0}$

- g_i, p_i , 一個delay ; $*$ 一個delay ; $+$ 一個delay

- Total (still) 3 delays

- $C_{in2} = g_1 + p_1 * g_0 + p_1 * p_0 * C_{in0}$

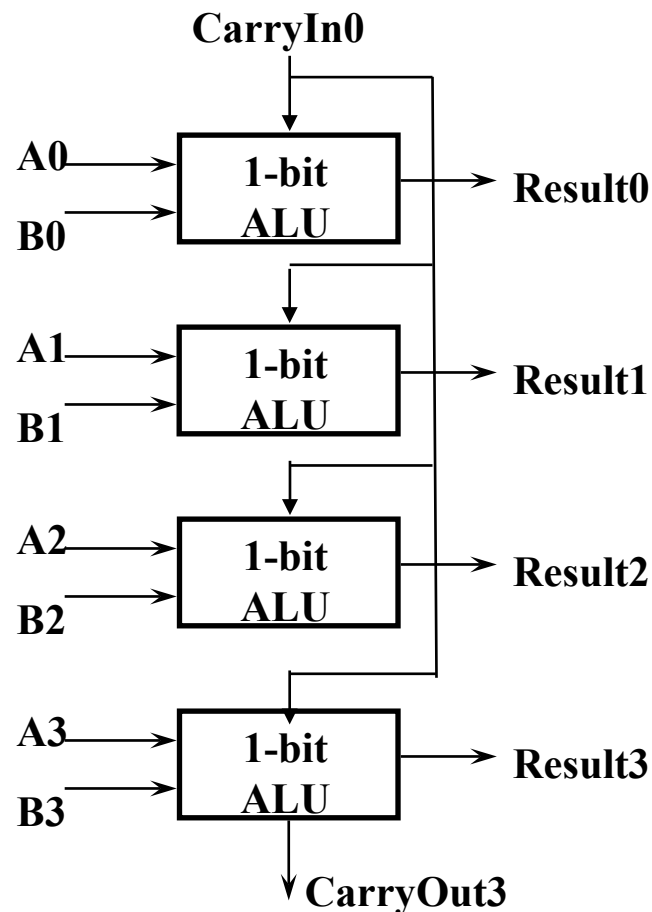
- Total (still) 3 delays

- $C_{in3} = g_2 + p_2 * g_1 + p_2 * p_1 * g_0 + p_2 * p_1 * p_0 * C_{in0}$

- Almost **constant delays** for C_{in} 's

Carry Lookahead Adder

- ❑ No Carry bit propagation from LSB to MSB
 - Inputs to an ALU's will also connect to latter bits' ALU's

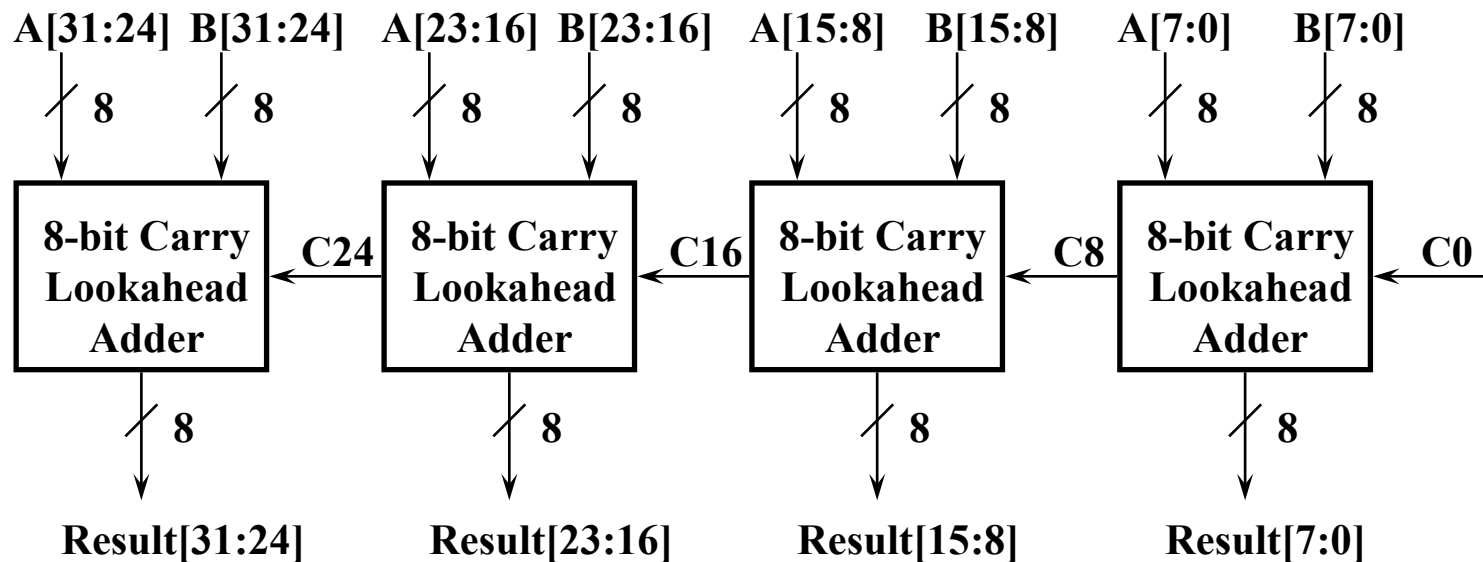


Common Carry Lookahead Adder

- ❑ Expensive to build a “full” carry lookahead adder
 - Just imagine length of the equation for C_{in31} (線路複雜)
- ❑ Common practices
 - Build smaller N-bit carry lookahead adders and then connect them together to form a bigger adder
 - Cascaded carry look-ahead adder
 - Multiple level carry look-ahead adder

Cascaded Carry Lookahead

- Connects several N-bit lookahead adders to form a big one



Multiple Level Carry Lookahead

□ View an N-bit lookahead adder as a block

- Generate “super” P_i and G_i of the block
- Use next level carry lookahead structure to generate block C_{in}

$$C_{in4} = g_3 + (p_3 \cdot g_2) + (p_3 \cdot p_2 \cdot g_1) + (p_3 \cdot p_2 \cdot p_1 \cdot g_0) + (p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot C_{in0})$$

Let (再設定更大一層的 lookahead: G_0, P_0)

$$G_0 = g_3 + (p_3 \cdot g_2) + (p_3 \cdot p_2 \cdot g_1) + (p_3 \cdot p_2 \cdot p_1 \cdot g_0)$$

$$P_0 = p_3 \cdot p_2 \cdot p_1 \cdot p_0$$

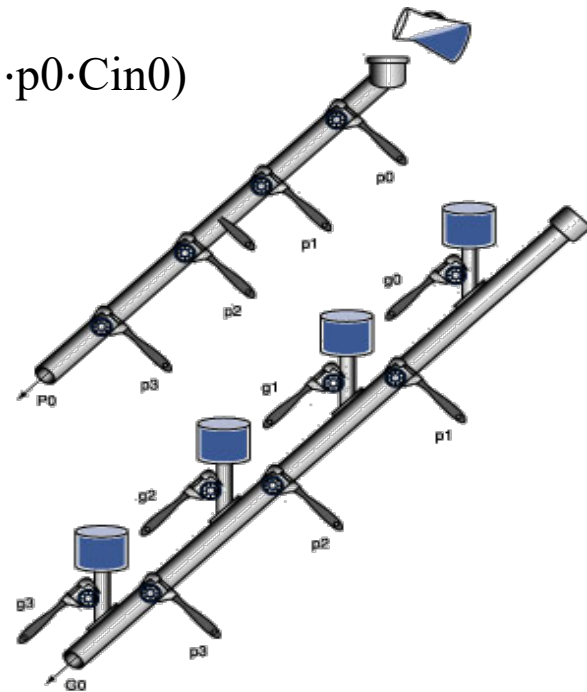
Currently 3-Gate delays

$$C_{in4} = G_0 + P_0 \cdot C_{in0}$$

Total $3 + 1 = 4$ gate delays

Generalize:

$$C_{in_{i+1}} = G_i + P_i \cdot C_{in_i}, \text{ } i \text{ is the number of 8-bit CLA block.}$$



Next Level Carry Lookahead

- ❑ Carry out equations for each 4-bit adder

$$C1 = G0 + (P0 \cdot c0)$$

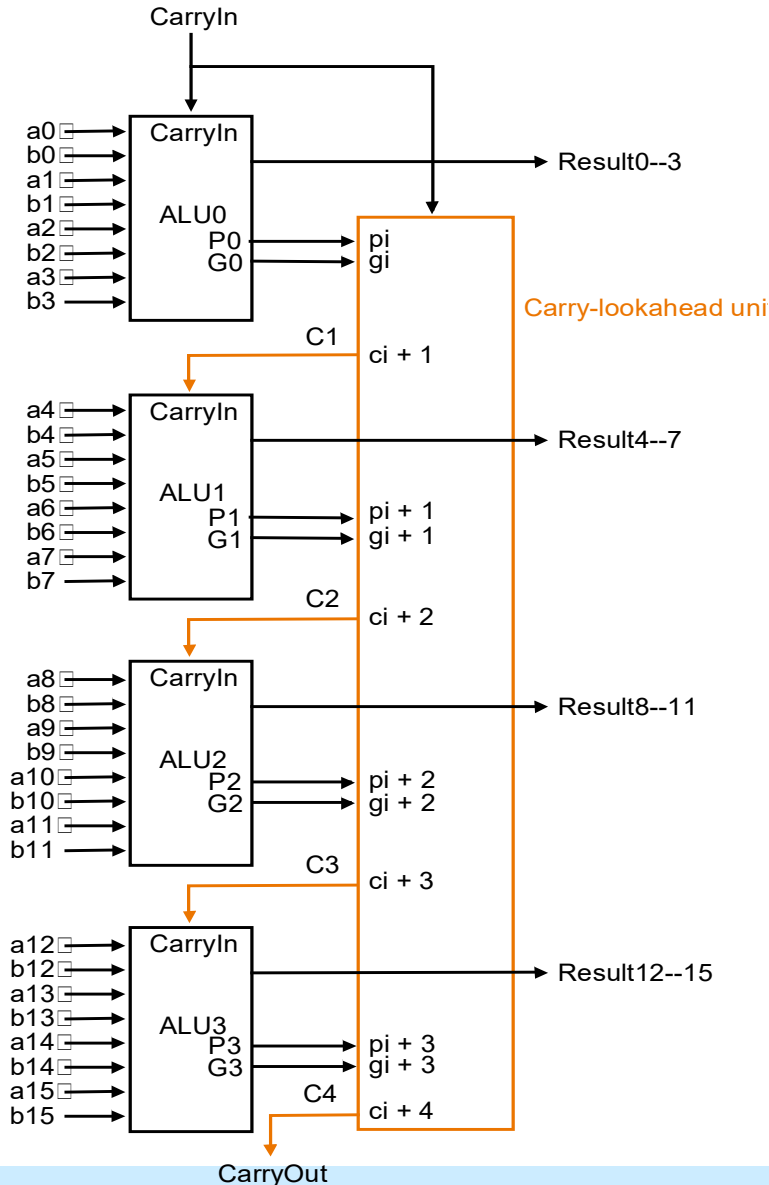
$$C2 = G1 + (P1 \cdot G0) + (P1 \cdot P0 \cdot c0)$$

$$C3 = G2 + (P2 \cdot G1) + (P2 \cdot P1 \cdot G0) + (P2 \cdot P1 \cdot P0 \cdot c0)$$

$$C4 = G3 + (P3 \cdot G2) + (P3 \cdot P2 \cdot G1) + (P3 \cdot P2 \cdot P1 \cdot G0) + (P3 \cdot P2 \cdot P1 \cdot P0 \cdot c0)$$

- Similar to the carry out equations for each bit of the 4-bit adder
- ❑ Also constant delay for computing C_i , since G_i and P_i for all i can be found in parallel with the same gate delay

A Multiple Level Carry Lookahead Adder



$$G = A * B$$

$$P = A + B$$

$$Cin_{i+1} = G_i + (P_i * Cin_i)$$

A	B	Cout	
0	0	0	kill (不會產生、停止)
0	1	Cin	propagate (看下一級、往下傳)
1	0	Cin	propagate (看下一級、往下傳)
1	1	1	generate (會產生)

Arithmetic for Multimedia

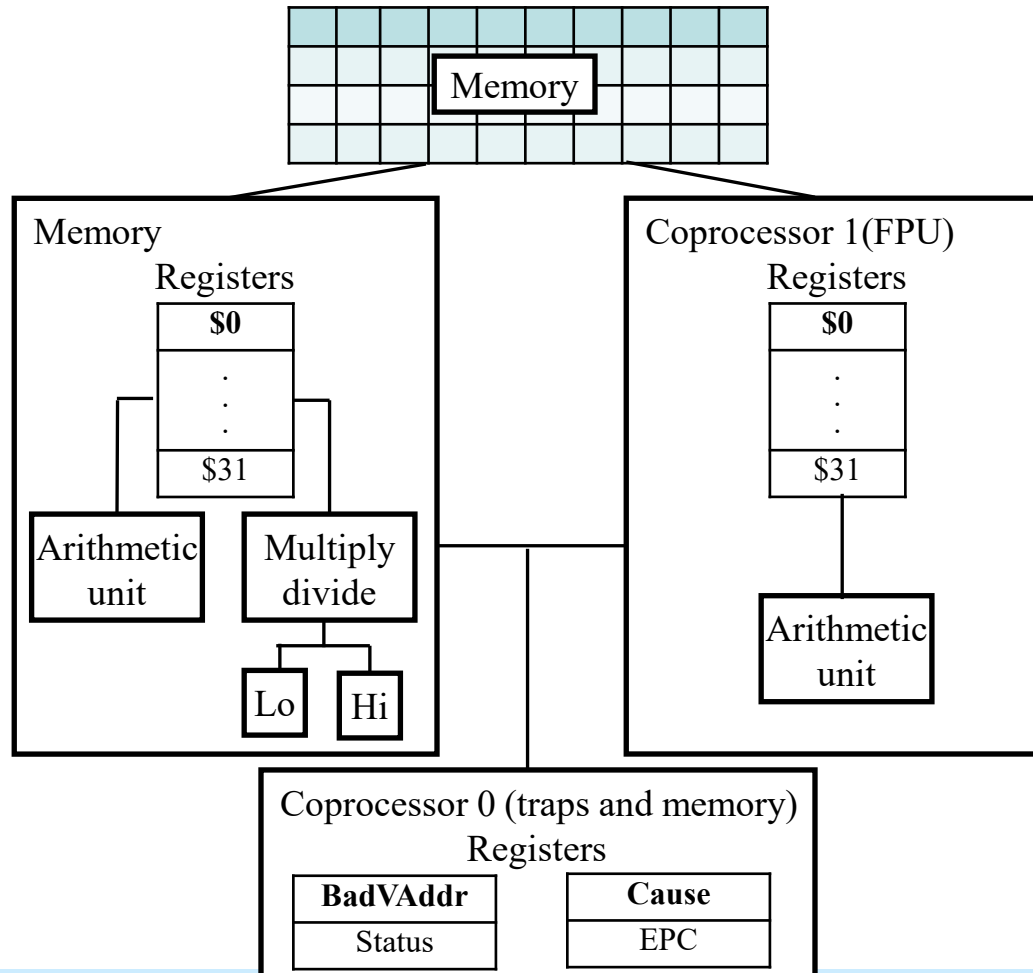
- Graphics and media processing operates on vectors of 8-bit and 16-bit data (多媒體算術運算的應用)
- Use 64-bit adder, with partitioned carry chain
 - Operate on 8×8-bit, 4×16-bit, or 2×32-bit vectors
- SIMD (**single-instruction, multiple-data**)
 - Intel 公司 MMX(MultiMedia eXtensions) 多媒體增強指令集技術實現單道指令多道資料流(SIMD)執行模式

□ Saturating operations

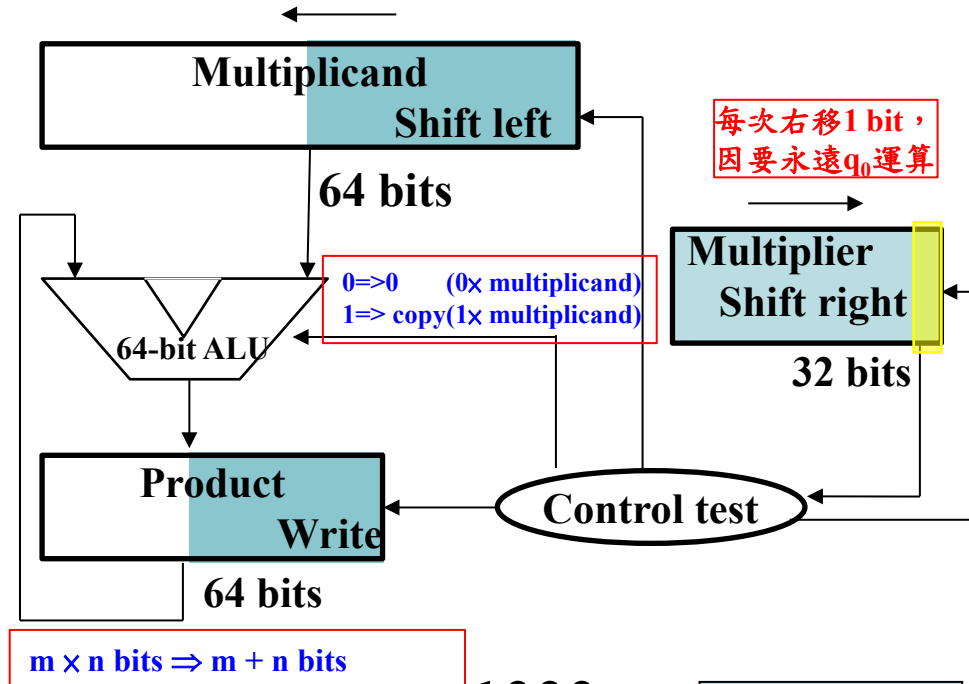
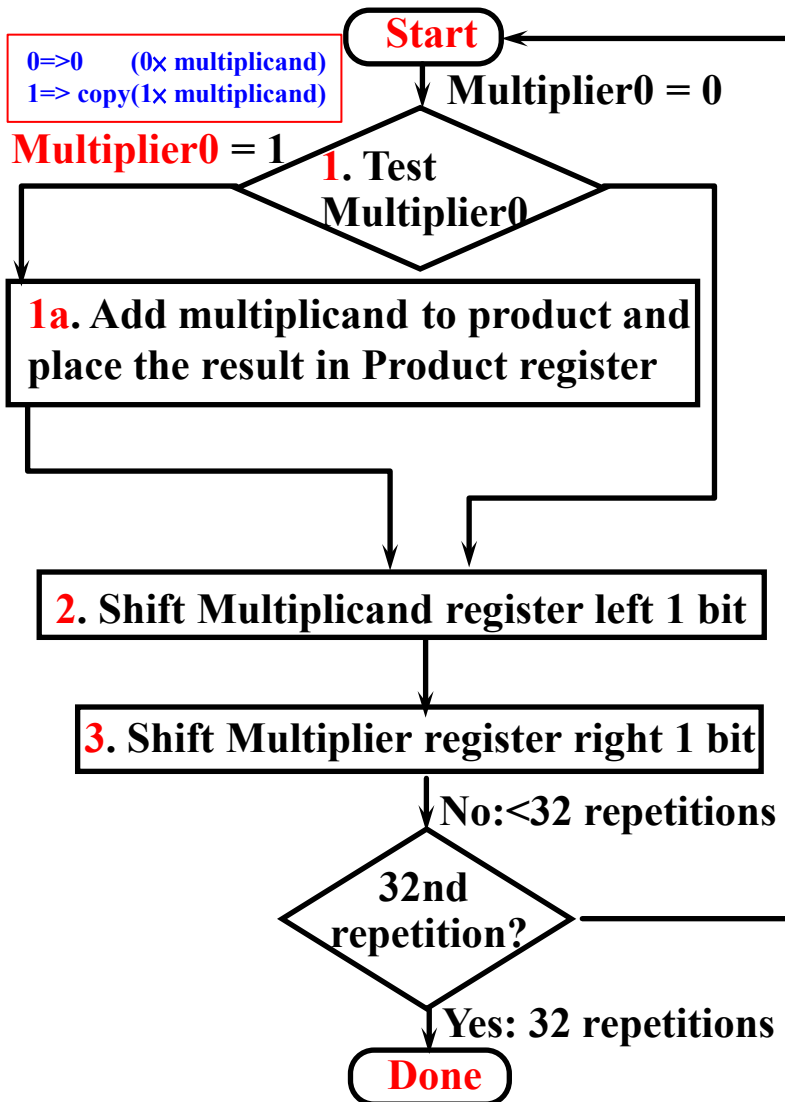
- On overflow, result is largest representable value
 - 飽和模式，若計算結果 overflow，CPU 自動去掉溢出，使計算結果取該資料型別表示上限值(上溢)或下限值(下溢)，一般用於 image 處理。
 - 2s-complement modulo arithmetic(overflow 會變成負數)
- e.g., clipping in audio (消去聲音波形上下), saturation in video

3.3 Multiplication

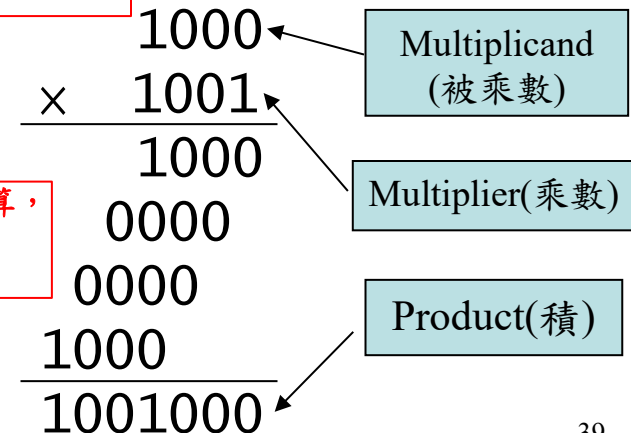
- ❑ Start with long-multiplication approach
 - 2 versions of multiply hardware and algorithm



Multiplication Algorithm Version 1

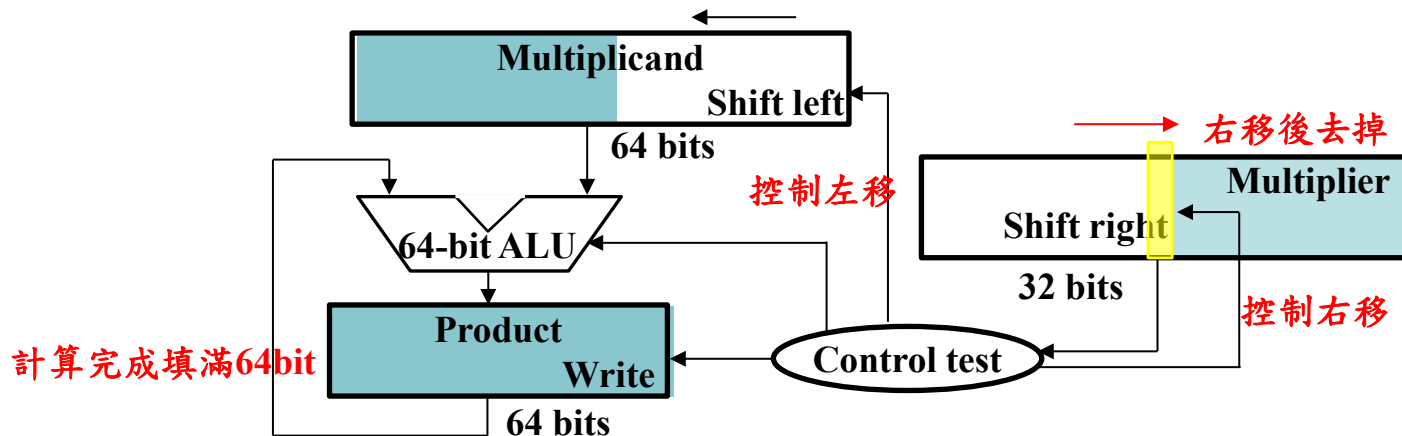
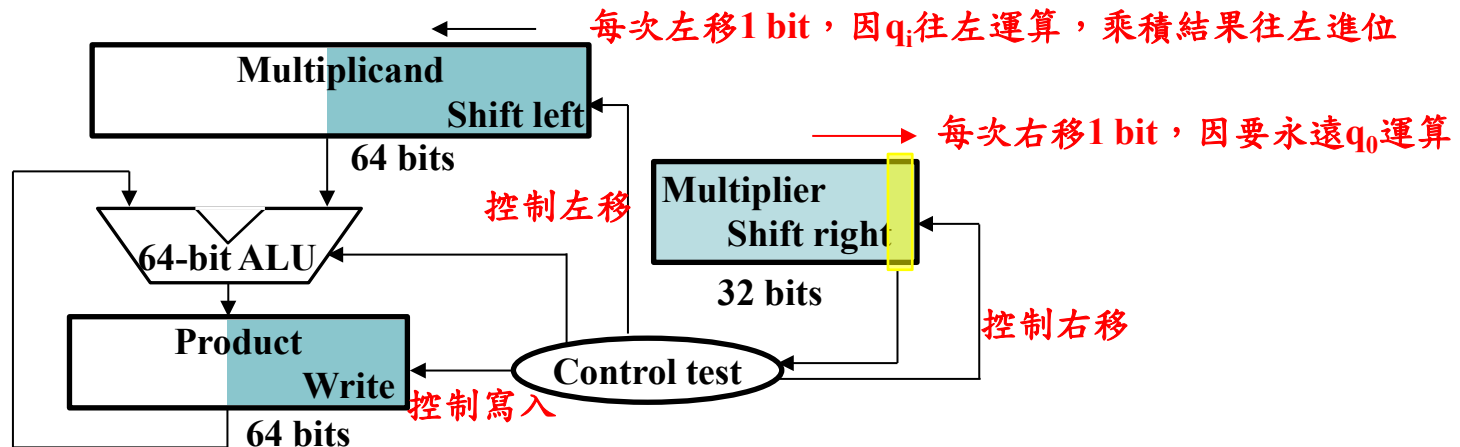


每次左移1 bit，因 q_i 往左運算，被乘數往左移位，乘積結果才能往左移位

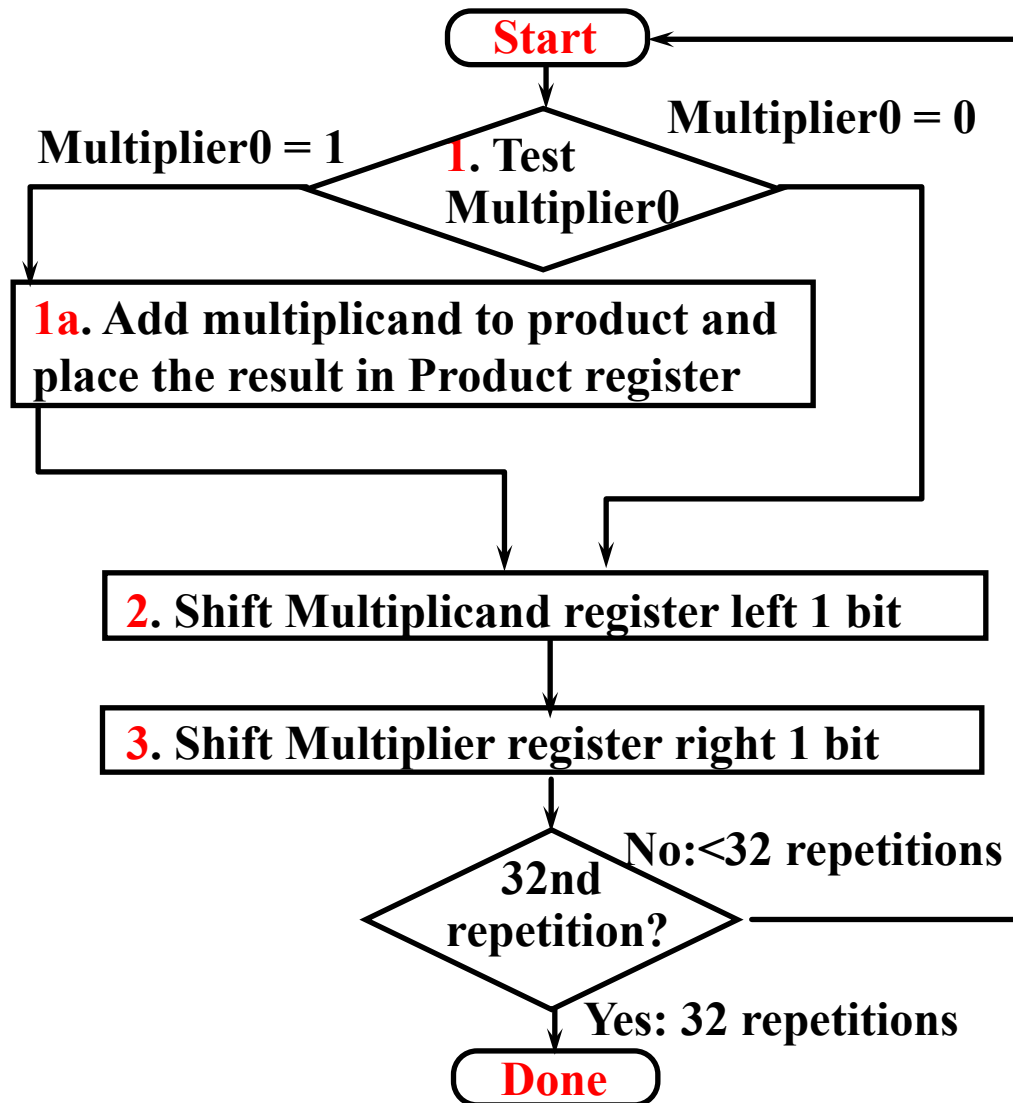


Multiplication Algorithm Version 1

- 64-bit multiplicand(被乘數) register (32-bit multiplicand at right half), 64-bit ALU, 64-bit product register, 32-bit multiplier register



Multiplication Algorithm Version 1



Example: 0010 × 0011

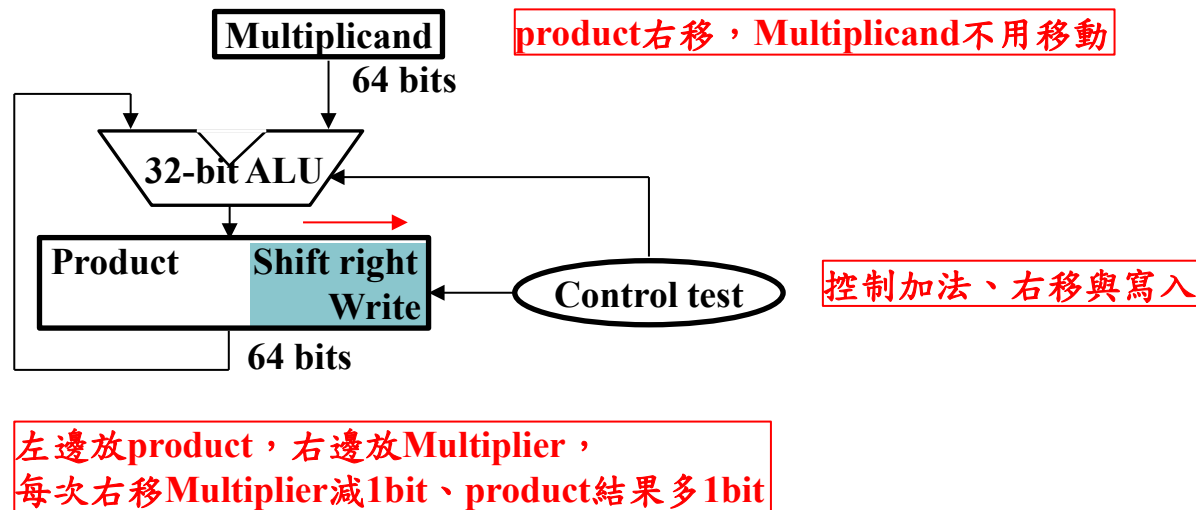
Product	Multiplier	Multiplicand
0000 0000	<u>0011</u>	0000 <u>0010</u> (S, 1)
0000 0010	<u>0001</u>	0000 <u>0100</u> (1a, 2, 3, 1)
0000 0110	<u>0000</u>	0000 <u>1000</u> (1a, 2, 3, 1)
0000 0110	<u>0000</u>	<u>0001</u> 0000 (2, 3, 1)
0000 0110	0000	<u>0010</u> 0000 (2, 3, D)

Observations for Multiplication Version 1

- ❑ 1 clock per cycle => ~ 100 ($= \text{start} + 32 * 3$) clocks per multiply
 - 一般程式中，加減法使用頻率，約為乘法5~100倍。
- ❑ 64-bit adder, multiplicand 浪費空間
 - 每次只有32bit加法運算
 - multiplicand左移後插入0，一半bit 永遠 0
- ❑ 64-bit Product register wastes space
 - product 一開始為 0
 - product後面bit計算出後就不會改變
- ❑ 整合 Multiplier and Product register
 - product暫存器右移(之前運算退一位)，取代multiplicand左移
 - Multiplier暫存器右移後左邊空出0，剛好給product暫存器使用

Optimized Multiplication

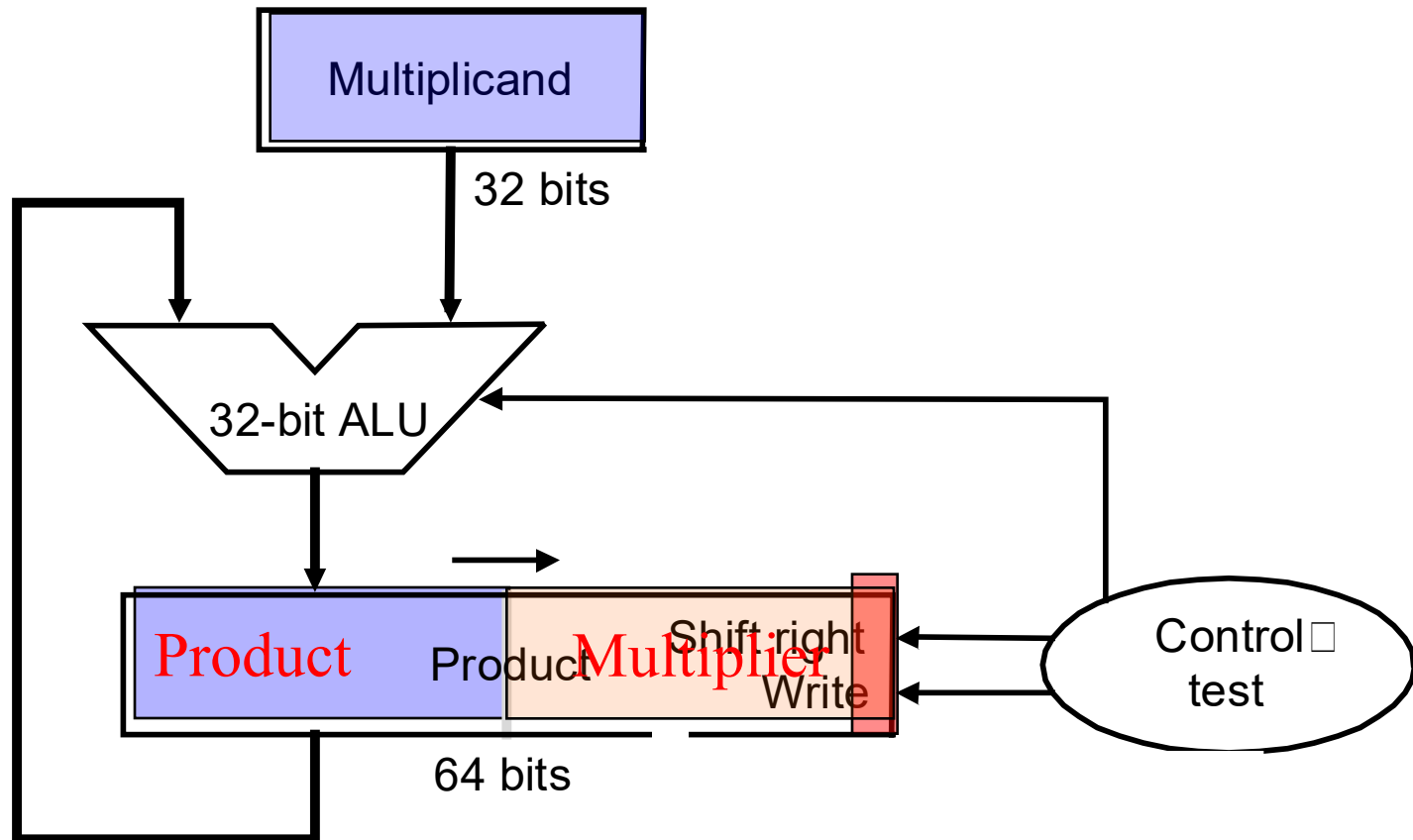
□ 改良後設計



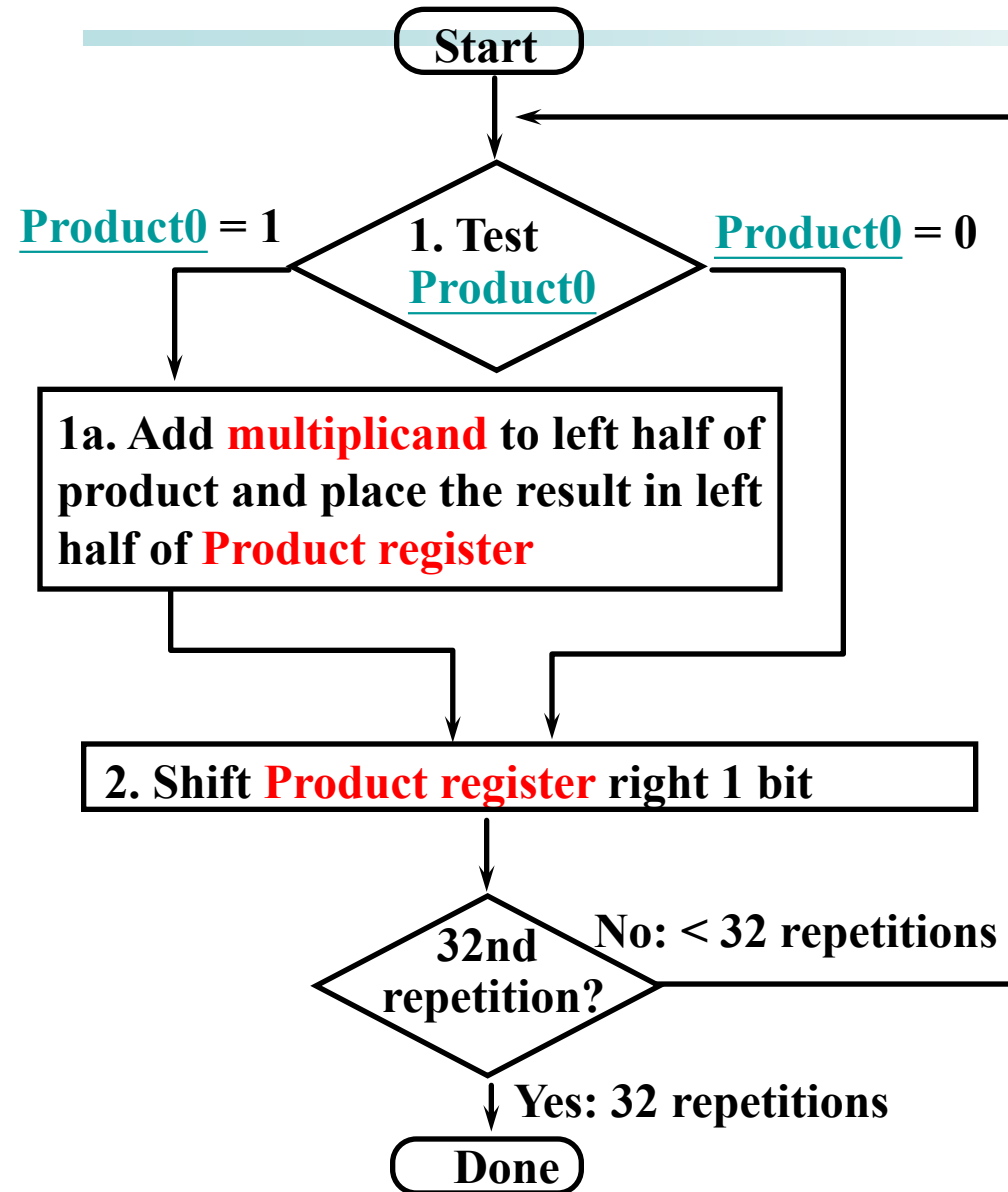
- Perform steps in parallel: add/shift
 - One cycle per partial-product addition

Multiplication Algorithm Version 2

- 32-bit Multiplicand register, 32-bit ALU, 64-bit Product register (HI & LO in MIPS), (0-bit Multiplier register, which is merged within the product register initially)



Multiplication Algorithm Version 2



Example: 0010 × 0011

Multiplicand	Product
0010	0000 <u>0011</u> (S, 1)
	0010 0011 (1a)
0010	0001 <u>0001</u> (2, 1)
	0011 0001 (1a)
0010	0001 <u>1000</u> (2, 1)
0010	0000 <u>1100</u> (2, 1)
0010	0000 <u>0110</u> (2, D)

S: Start
1a: step 1a
D: Done

Signed Multiplication

❑ Signed multiplication

- Make both positive and remember whether to complement product when done(不論正負號，全部轉成正數；計算完後根據正負號，將乘積做補數轉換)
 - leave out sign bit, run for 31 steps, since it must be 0 during multiplication in this method(省略正負號bit，執行31步乘法，因為乘法時均為正數)
- Booth's Algorithm

Booth's Algorithm

□ 8bit, $\underline{-7} \times \underline{4}$

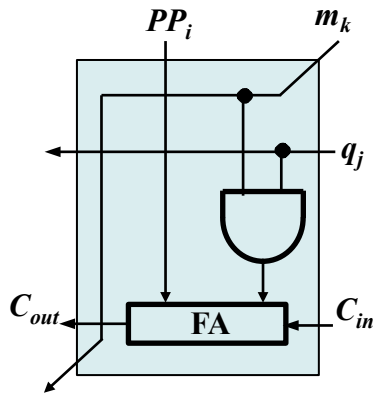
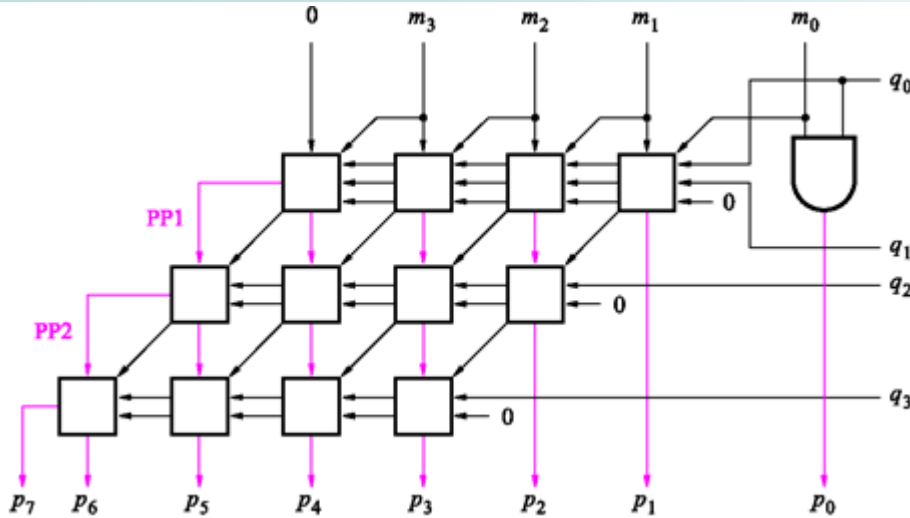
- 2'補數，被乘數11111001 與乘數00000100
- 乘數最右補0，右至左相鄰2bit以右表格運算，得 00001(-1)00
- 若被乘數M為11111001，-M為00000111。
- 將11111001與剛運算得00001(-1)00相乘
 - 乘數為1時複製M
 - 乘數為-1時複製-M
 - 最後相加

$$\begin{array}{r}
 11111001 \\
 \times 00001-100 \\
 \hline
 0 \\
 0 \\
 00000111 \quad (-M) \\
 11111001 \quad (M) \\
 \hline
 11111100100 = -28
 \end{array}$$

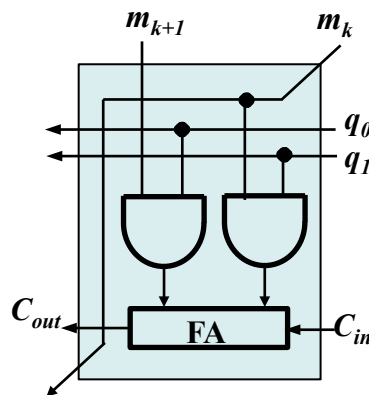
$$\begin{array}{r}
 11111001 \\
 \times 00000100(0) \\
 \hline
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 0 \\
 00001(-1)00
 \end{array}$$

00	0
01	1
10	-1
11	0

陣列乘法器



下兩列區塊_{ut} (元件B)



最上列區塊_{ut} (元件A)

$$\begin{array}{r}
 m_3 m_2 m_1 m_0 \\
 1110 \\
 \times 1011 \\
 \hline
 1110 \\
 + 1110 \\
 \hline
 10101 \\
 + 0000 \\
 \hline
 01010 \\
 + 1110 \\
 \hline
 10011010 \\
 \text{P}_2 \text{P}_1 \text{P}_0
 \end{array}$$

被乘數 m_i
乘數 q_i

部分乘積(元件A)
 q_0 跟 m_{k+1} 做and
 q_1 跟 m_k 做and再相加

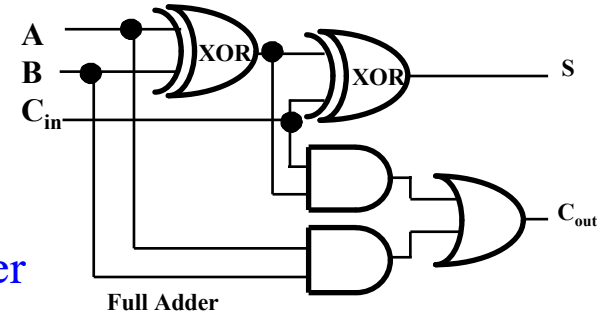
部分乘積(元件B)
 q_i 跟 m_k 做and再相加

Wallace Tree Multiplier

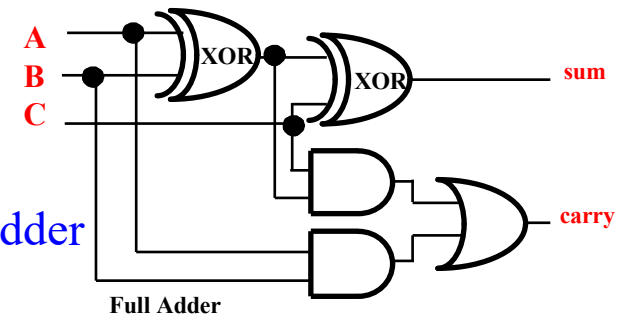
- Use **carry save adders (CSA)**

1 0 1 0 1 1 1 0	A
0 0 1 0 0 0 1 1	B
1 0 0 0 0 1 1 1	C
0 0 0 0 1 0 1 0	(sum)
1 0 1 0 0 1 1 1	(carry)

regular adder



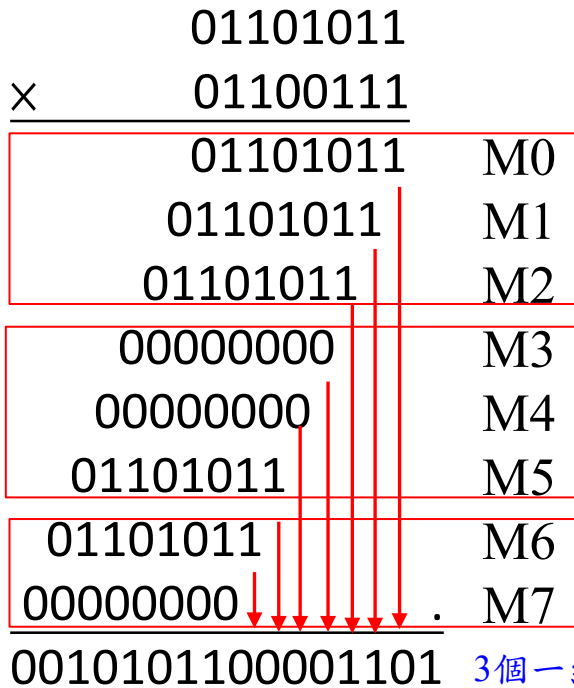
carry save adder



- three inputs and two outputs
 - 8 bits carry save adder: 8 full adders for 8 bits
 - One full adder delay (**no carry propagation**)
 - The last stage is performed by **regular adder**
- What is the minimum delay for **16 x 16 multiplier** ?

8-bit Wallace Tree Multiplier

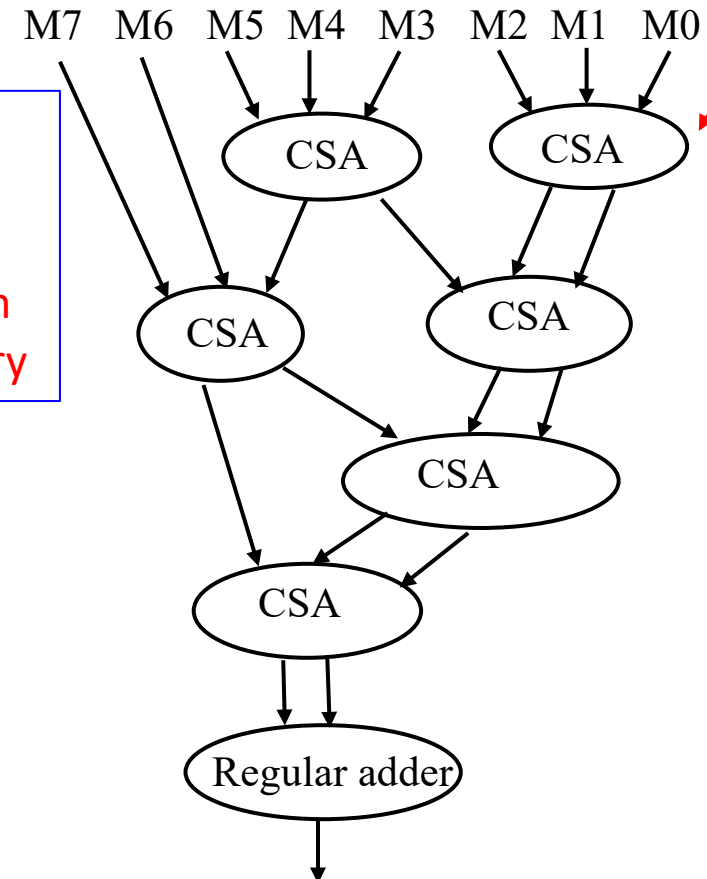
$$107 \times 103 = 11021$$



3個一組
最後未滿3個則2個一組

01101011 M0
01101011 M1
01101011 M2
0100010001 sum
0011101110 carry

每一層可以平行處理
第一層運算完成換到下一層，
可以使用pipeline



MIPS Multiplication

- ❑ Two 32-bit registers for product
 - HI: most-significant 32 bits
 - LO: least-significant 32-bits
- ❑ Instructions
 - mult rs, rt / multu rs, rt
 - 64-bit product in HI/LO
 - mfhi rd / mflo rd
 - Move from HI/LO to rd
 - Can test HI value to see if product overflows 32 bits
 - mul rd, rs, rt
 - Least-significant 32 bits of product → rd

Multiplication in MIPS

- ❑ No destination register: product could be $\sim 2^{64}$; need two special registers to hold it

- 3-step process:

mult \$t1, \$t2 # t1 * t2

\$t1

0**11111111111111111111111111111111**

$\times$ *\$t2*

0**1**00000000000000000000000000000000

000**11111111111111111111111111111111**11000000000000000000000000000000

Hi

Lo

mfhi \$t3

\$t3

000**11111111111111111111111111111111**

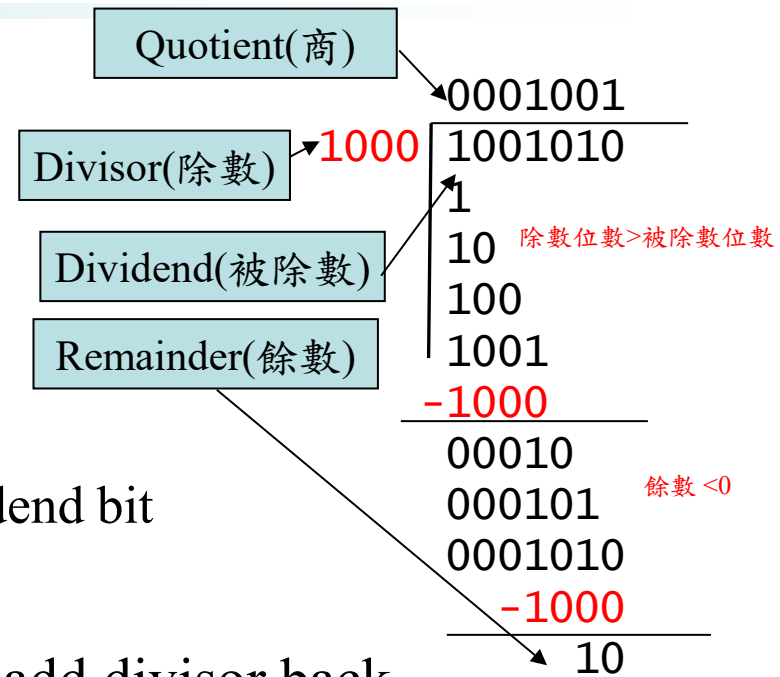
mflo \$t4

\$t4

1100000000000000000000000000000000

3.4 Division

- ❑ Check for 0 divisor (軟體檢查)
- ❑ Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- ❑ Restoring division
 - Do the subtract, and if remainder < 0 , add divisor back



❑ Signed division

- Divide using absolute values
- Adjust sign of quotient and remainder as required

實做方法

- 1.1 餘數暫存器設定= ≥ 2 倍有效位
- 1.2 被除數放在餘數暫存器最右邊
- 2.1 除數暫存器設定= ≥ 2 倍有效位
- 2.2 除數放除數暫存器最左邊，每次右移一位

```
while (還有下一位數被除數)
if 除數位數  $\leq$  被除數位數
    商為 1
    餘數 = 被除數 - 除數
if 餘數  $< 0$ 
    被除數 = 被除數 + 除數
    商為 0
```

Divide Algorithm Version 1

Example: 0111 / 0010

Start: Place **Dividend** in **Remainder**

1. **Subtract Divisor register** from **Remainder register**, and place the result in **Remainder register**

Remainder ≥ 0 Test Remainder Remainder < 0

2a. Shift **Quotient** register to left, setting new rightmost bit to **1**

2b. **Restore** original value by adding **Divisor** to **Remainder**, place sum in **Remainder**, shift **Quotient** to the left, setting new least significant bit to **0**

3. Shift **Divisor** register right 1 bit

33rd repetition? No: < 33 repetitions

Done

Yes: 33 repetitions

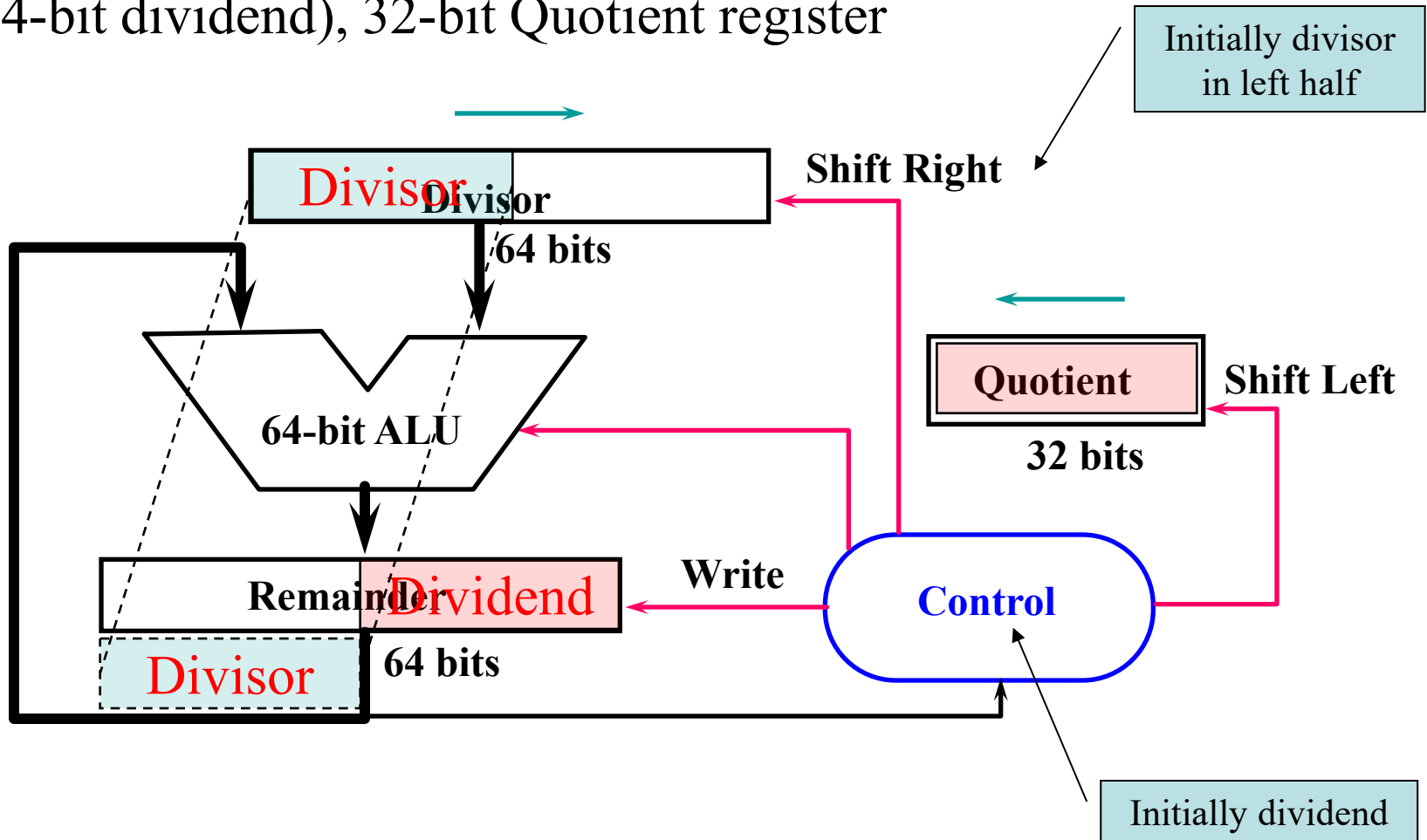
Quot.	Divisor	Remainder
0000	00100000	0000 0111 (S)
		11100111 (1)
0000		00000111 (2b)
0000	00010000	0 0000111 (3)
		11110111 (1)
0000		00000111 (2b)
0000	00001000	00 000111 (3)
		11111111 (1)
0000		00000111 (2b)
0000	00000100	000 00111 (3)
		00000011 (1)
0001		00000011 (2a)
0001	00000010	0000 0011 (3)
		00000001 (1)
00011		00000001 (2a)
0011	00000001	00000001 (3,D)

00000111
+11100000
11100111

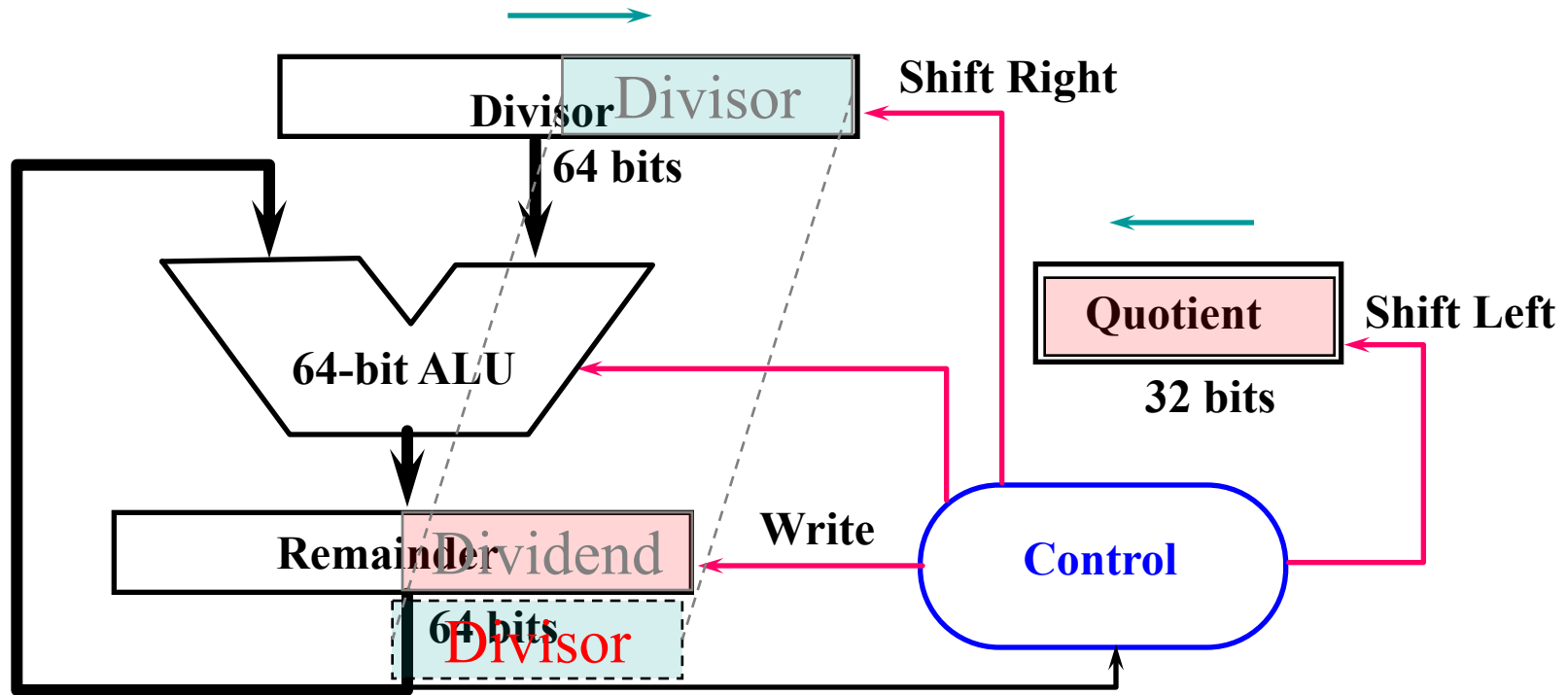
00000111(Start)
- 00100000
11100111(步驟1計算)

Divide Hardware Version 1

- 64-bit Divisor register (initialized with 32-bit divisor in left half), 64-bit ALU, 64-bit Remainder register (initialized with 64-bit dividend), 32-bit Quotient register



Divide Hardware Version 1



Observations: Divide Version 1

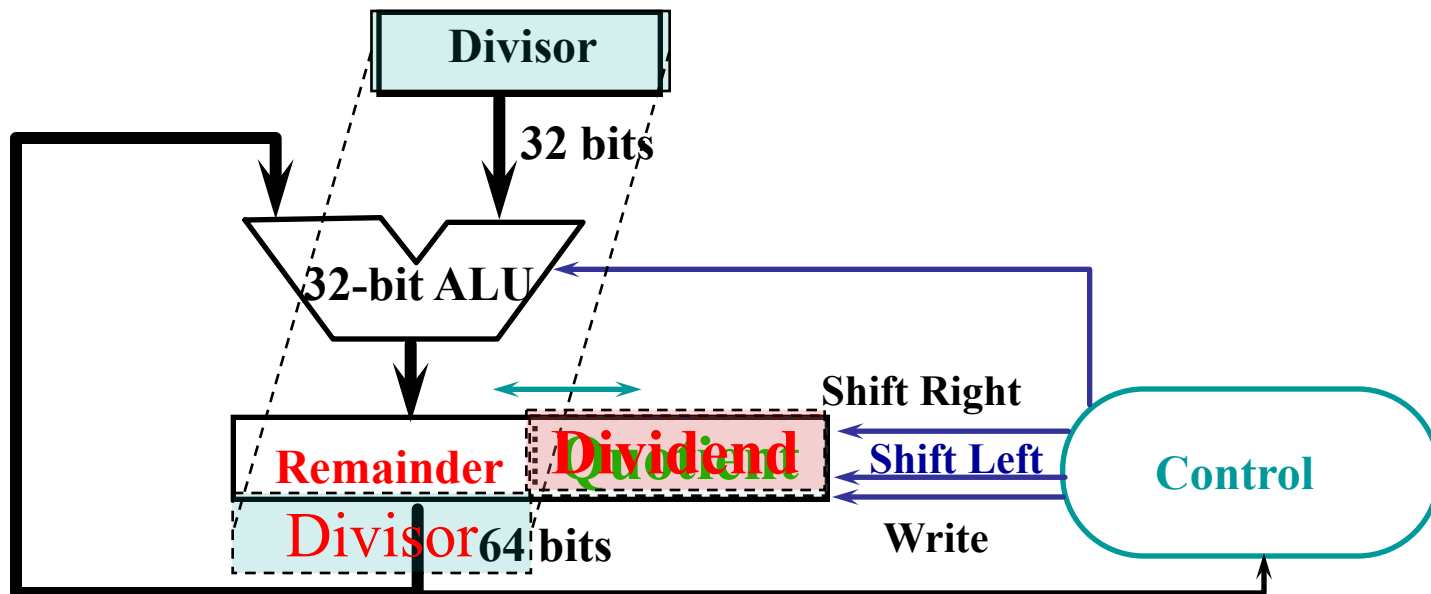
- ❑ Half of the bits in divisor register always 0
 - => 1/2 of 64-bit adder is wasted
 - => 1/2 of divisor is wasted
- ❑ Instead of shifting divisor to right, shift remainder to left?
- ❑ Eliminate Quotient register by combining with Remainder register as shifted left
 - Remainder往左移，右邊空出給quotient，只用一半bit減divisor，最後remainder取左半bit當remainder結果，右半bit是quotient
- ❑ 1st step cannot produce a 1 in quotient bit (otherwise quotient is too big for the register)
 - => switch order to shift first and then subtract
 - => save 1 iteration

以4bit為例，極端情況1111/0001，
第一次quotient不可能為1，
直接先shift再減，省掉一次反覆。

00001111(Start)
- 00010000 .
11100111(步驟1計算)

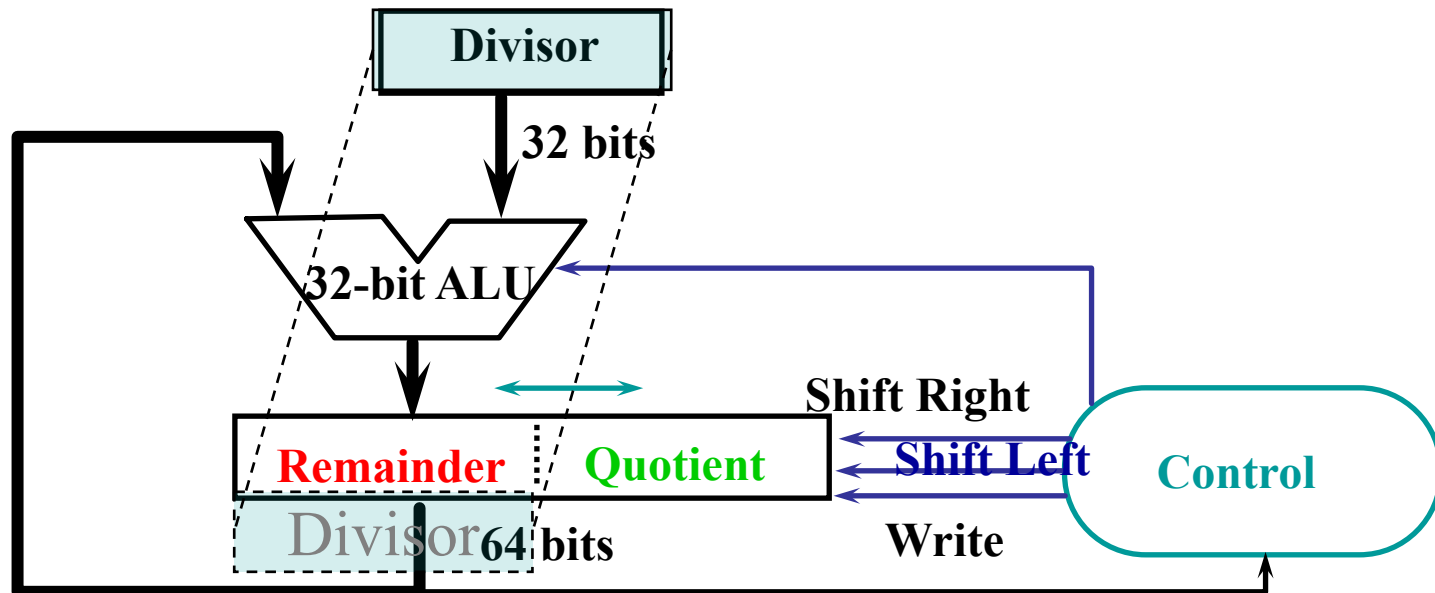
Divide Hardware Version 2

- ❑ 32-bit Divisor register, 32-bit ALU, 64-bit Remainder register, (0-bit Quotient register)
 - like multiplier (Same hardware can be used for both)



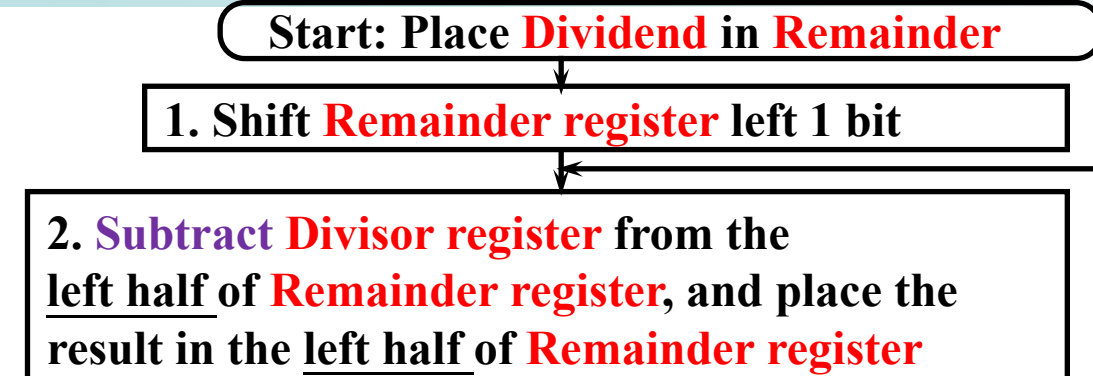
- One cycle per partial-remainder subtraction(每個cycle半個remainder 減法)

Divide Hardware Version 2



Divide Algorithm Version 2

Example: 0111 / 0010



Remainder ≥ 0

Remainder < 0

Test
Remainder

3a. Shift **Remainder** to left, setting new rightmost bit to **1**

3b. Restore original value by adding **Divisor** to left half of **Remainder**, and place sum in left half of **Remainder**. Also shift **Remainder** to left, setting the new least significant bit to **0**

32nd
repetition?

No: < 32 repetitions

Yes: 32 repetitions

Done. Shift left half of **Remainder** right 1 bit

Step	Remainder	Divs.
0 S	0000 0111	0010
1.1	0000 1110	
1.2	1110 1110	(減)
1.3b	0001 1100	(左移)
2.2	1111 1100	
2.3b	0011 1000	
3.2	0001 1000	
3.3a	0011 0001	
4.2	0001 0001	
4.3a	0010 0011	
4.D	0001 0011	

Signed Divide

❑ Signed Divides

- Remember signs, make positive, complement quotient and remainder if necessary

- Let Remainder and Dividend have same sign
- Negate Quotient if Divisor sign & Dividend sign disagree
- E.g., $-7 \div 2 = -3$, remainder = -1
- $-7 \div -2 = 3$, remainder = -1
- Satisfy $\text{Dividend} = \text{Quotient} \times \text{Divisor} + \text{Remainder}$

❑ Possible for quotient to be too large:

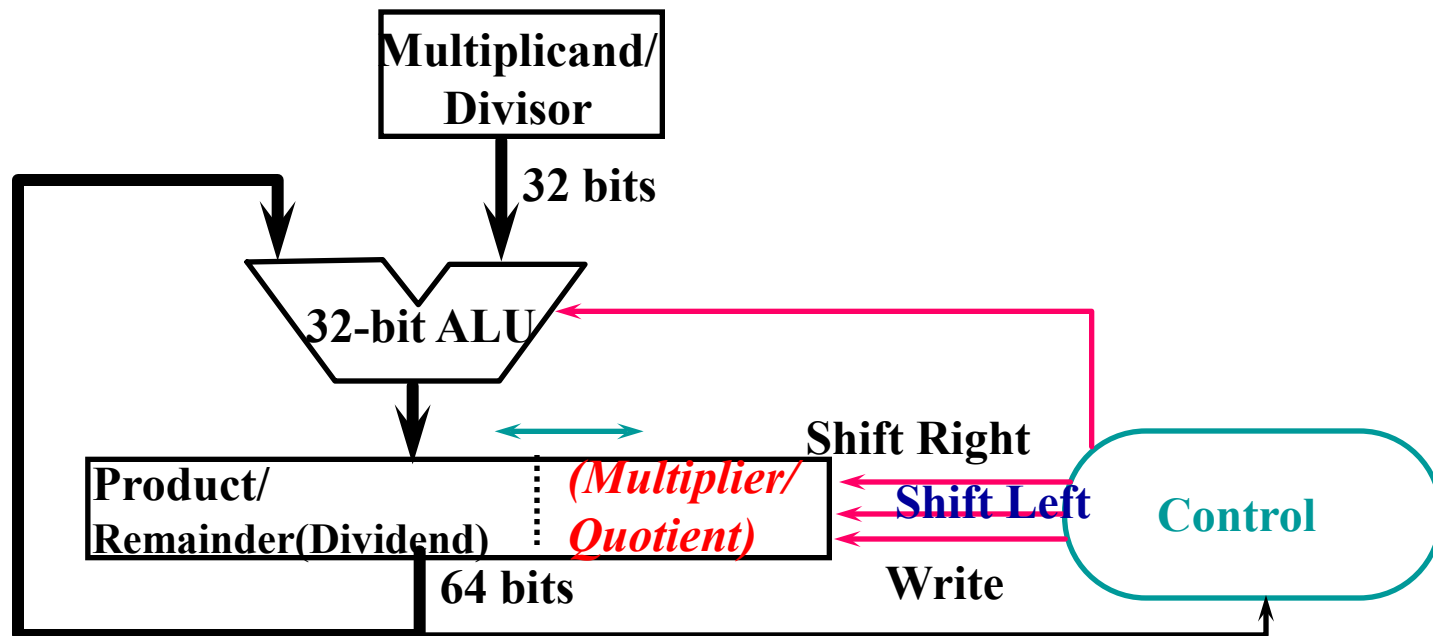
if divide 64-bit integer by 1, quotient is 64 bits

Observations: Multiply and Divide

- ❑ Same hardware as multiply: just need ALU to add or subtract, and 64-bit register to shift left or shift right
- ❑ Hi and Lo registers in MIPS combine to act as 64-bit register for multiply and divide

Multiply/Divide Hardware

- 32-bit Multiplicand/Divisor register, 32-bit ALU, 64-bit Product/Remainder register, (0-bit Multiplier/Quotient register)



Faster Division

- ❑ Can't use parallel hardware as in multiplier
 - Subtraction is conditional on sign of remainder
 - 每次運算正負號不確定，無法像乘法器一樣平行硬體執行
- ❑ Faster dividers (e.g. SRT division) generate multiple quotient bits per step
 - 嘗試每一步驟預測多個quotient bit
 - 使用好的lookup table，查詢前面dividend和remainder的bit，以預測quotient bit
 - Algorithm 使用6 bits remainder和4 bits divisor
 - 若預測錯誤再逐步修正
 - Still require multiple steps
 - [http://en.wikipedia.org/wiki/Division_\(digital\)](http://en.wikipedia.org/wiki/Division_(digital))

MIPS Division

- ❑ Use HI/LO registers for result
 - HI: 32-bit remainder
 - LO: 32-bit quotient

- ❑ Instructions
 - `div rs, rt` / `divu rs, rt`
 - No overflow or divide-by-0 checking
 - Software must perform checks if required
 - Use `mfhi`, `mflo` to access result

Division in MIPS

- ❑ Quotient stored in Lo, remainder in Hi

div \$t1, \$t2 # $t1 / t2$

mflo \$t3 #copy quotient (lo) to t3

mfhi \$t4 #copy remainder (hi) to t4

- ❑ Unsigned multiplication and division:

multu \$t1, \$t2 # $t1 * t2$

divu \$t1, \$t2 # $t1 / t2$

- Just like mult, div, except now interpret t1, t2 as unsigned integers instead of signed
- Answers are also unsigned, use mfhi, mflo to access

MIPS Multiply/Divide Summary

❑ Start multiply, divide

- mult rs, rt # HI:LO = rs $\times$ rt ; 64-bit signed
- multu rs, rt # HI:LO = rs $\times$ rt ; 64-bit unsigned
- div rs, rt # LO = rs $\div$ rt; HI = rs mod rt
- divu rs, rt

❑ Move result from multiply, divide

- nfhi rd # rd = HI
- mflo rd # rd = LO

❑ Move to HI or LO

- mthi rd # HI = rd
- mtlo rd # LO = rd

3.5 Floating Point

- ❑ Representation for non-integral numbers
 - Including very small and very large numbers
- ❑ Like scientific notation
 - -2.34×10^{56} ← normalized
 - $+0.002 \times 10^{-4}$ ← not normalized
 - $+987.02 \times 10^9$ ← not normalized
- ❑ In binary
 - $\pm 1.\text{xxxxxxxx}_2 \times 2^{\text{yyyy}}$
- ❑ Types float and double in C

Floating Point Standard

- ❑ Defined by IEEE Std 754-1985

- Developed in response to divergence(分歧) of representations

- 可以運用於不同浮點數表示格式

- 被普遍採用，因此可改善科學浮點小數運算移植性問題(portability issues)

- Two representations

- Single precision (32-bit)

- Double precision (64-bit)

IEEE Floating-Point Format

- ❑ S: sign bit ($0 \Rightarrow$ non-negative, $1 \Rightarrow$ negative)
- ❑ Normalize significand: $1.0 \leq |\text{significand}| < 2.0$
 - leading pre-binary-point 1 bit, no need to represent it explicitly (**hidden bit**)
 - Significand is Fraction with the "1." restored
- ❑ Exponent: excess representation: actual exponent + Bias
 - Ensures exponent is unsigned (指數以正數表示)
 - Single: Bias = 127; Double: Bias = 1023 (表達很大和很小的數)

single: 8 bits

single: 23 bits

double: 11 bits

double: 52 bits

S	Exponent(8/11)	Fraction(Mantissa)(23/52)
---	----------------	---------------------------

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

Single-Precision Range

❑ Exponents 00000000 and 11111111 reserved

❑ Smallest value

○ Exponent: 00000001

actual exponent = $1 - 127 = -126$

○ Fraction: 000...00 $\Rightarrow$ significand = 1.0

○ $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$

❑ Largest value

○ exponent: 11111110

actual exponent = $254 - 127 = +127$

○ Fraction: 111...11 (≈ 1.0) $\Rightarrow$ significand $1.0 + (\approx 1.0) = (\approx 2.0)$

○ $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Double-Precision Range

- ❑ Exponents 0000...00 and 1111...11 reserved

- ❑ Smallest value

- Exponent: 00000000001

- actual exponent = $1 - 1023 = -1022$

- Fraction: 000...00 $\Rightarrow$ significand = 1.0

- $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$

- ❑ Largest value

- Exponent: 11111111110

- actual exponent = $2046 - 1023 = +1023$

- Fraction: 111...11 $\Rightarrow$ significand ≈ 2.0

- $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Floating-Point Precision

□ Relative precision

- all fraction(mantissa尾數) bits are significant
 - Single: Fraction = 1111...11(23bit) , 分別代表 $2^{-1}, 2^{-2}, 2^{-3}, \dots, 2^{-23}$
- Single: approximate 2^{-23} (取 $\log_{10}=10^{-6}$)
 - Equivalent to $23 \times \log_{10}2 \approx 23 \times 0.3 \approx 6$ decimal digits of precision
- Double: approximate 2^{-52}
 - Equivalent to $52 \times \log_{10}2 \approx 52 \times 0.3 \approx 16$ decimal digits of precision

Floating-Point Example

□ Represent $-0.75 = (-1)^1 \times 1.1_2 \times 2^{-1}$

○ $S = 1$, Fraction = $1000...00_2$, Exponent = $-1 + \text{Bias}$

➤ Single: $-1 + 127 = 126 = \underline{01111110}_2$

➤ Double: $-1 + 1023 = 1022 = \underline{01111111110}_2$

○ Single: $\underline{101111110}1000...00$

○ Double: $\underline{101111111110}1000...00$

□ **Exercise:** What number is represented by the single-precision float

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

○ $1\underline{10000001}01000...00$

S	Exponent(8/11)	Fraction(Mantissa)(23/52)
---	----------------	---------------------------

➤ $S = 1$, Fraction = $01000...00_2$, Exponent = $10000001_2 = 129$

$$x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$$

$$= (-1) \times 1.25 \times 2^2$$

$$= -5.0$$

Denormalized Number

□ Denormalized Number 定義

- Exponent = 000...0 $\Rightarrow$ (意謂) hidden bit is 0

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$$

□ Denormalized Number is smaller than normal numbers

- gradual underflow 定義: Fraction 部位逐漸趨近於0，但不為0

➤ Exponent 和 Fraction 均為0，定義為0

$$x = (-1)^S \times (0 + 0) \times 2^{-\text{Bias}} = \pm 0.0$$

- Denormal with fraction = 000...1 是最小值

0.0000 0000 0000 0000 0000 001 $\times 2^{-126}$, or 1.0×2^{-149}
一般single precision 最小值是 1.0×2^{-146}

2 種 0.0 表達方式
(所以 Denormal 的 Fraction 不能為0)

Single precision		double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	± 0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1-254	Anything	1-2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Infinites and NaNs

- ❑ Exponent = 111...1, Fraction = 000...0
 - $\pm$ Infinity
 - 提供後續計算使用，避免增加 overflow 檢查
- ❑ Exponent = 111...1, Fraction $\neq$ 000...0
 - Not-a-Number (NaN)
 - Indicates illegal or undefined result
 - e.g., 0.0 / 0.0
 - 提供後續計算使用

Floating-Point Addition

□ Consider a 4-digit decimal example

- $9.999 \times 10^1 + 1.610 \times 10^{-1}$

- 1. Align decimal points

 - Shift number with **smaller exponent**

- $9.999 \times 10^1 + 0.016 \times 10^1$

- 2. Add significands

- $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$

- 3. Normalize result & check for over/underflow

- 1.0015×10^2

- 4. Round and renormalize if necessary (**4-digit**)

- **1.002×10^2**

Floating-Point Addition

- consider a 4-digit binary example

- $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ ($0.5 + -0.4375$)

- 1. Align binary points

- Shift number with smaller exponent (比較小的要右移)

- $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$

- 2. Add significands

- $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$

- 3. Normalize result & check for over/underflow

- $1.000_2 \times 2^{-4}$, with no over/underflow

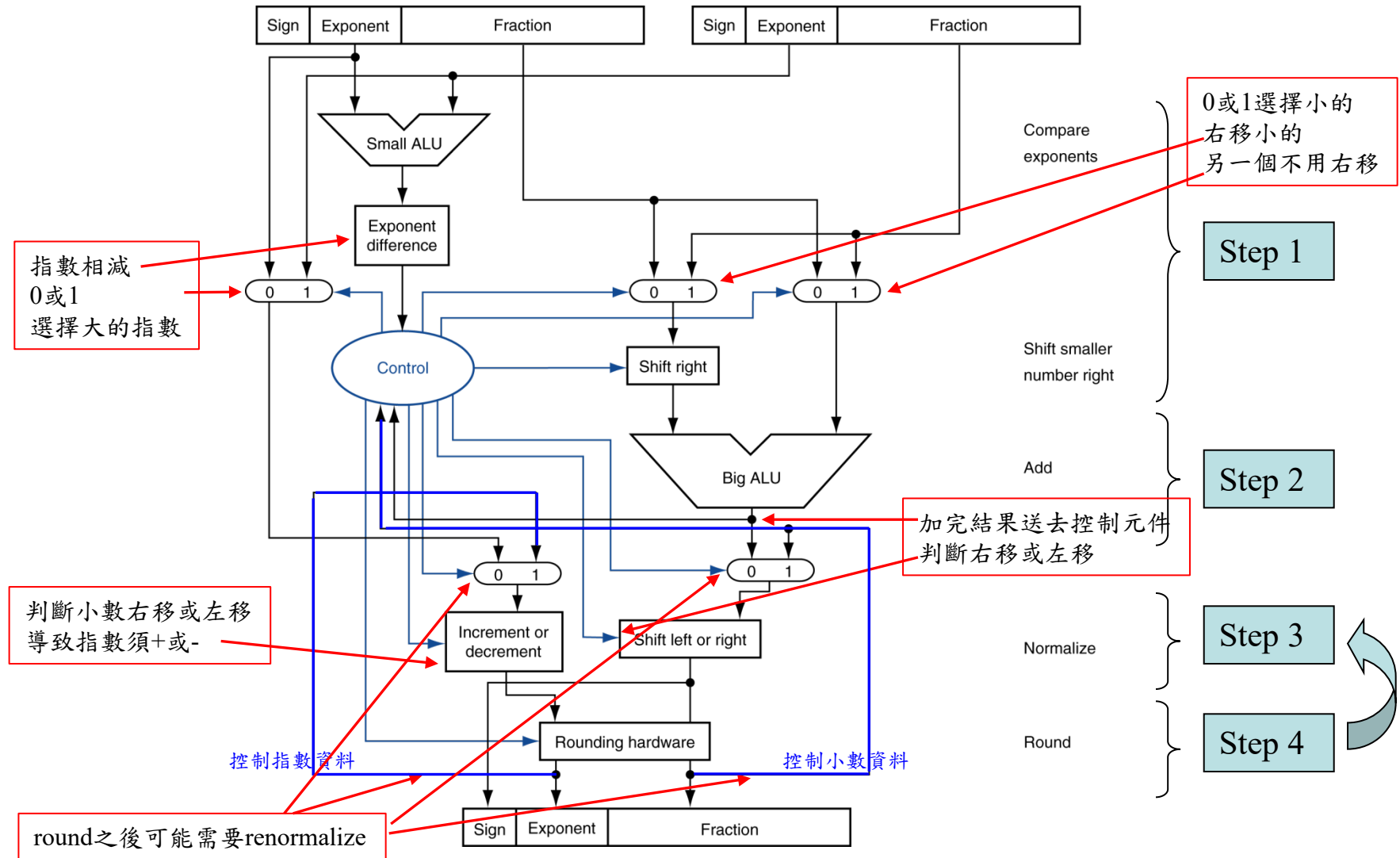
- 4. Round and renormalize if necessary

- $1.000_2 \times 2^{-4}$ (no change) = 0.0625

FP Adder Hardware

- ❑ Much more complex than integer adder
- ❑ Doing it in one clock cycle would take **too long**
 - Much longer than integer operations
 - Slower clock would penalize all instructions
 - 建置一個專責的processor FPU
- ❑ FP adder usually takes **several cycles**
 - Can be **pipelined**

FP Adder Hardware



Floating-Point Multiplication

- ❑ Consider a 4-digit decimal example
 - $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$
- ❑ 1. Add exponents
 - For biased exponents, **subtract** bias from sum
 - New exponent = $10 + -5 = 5$
- ❑ 2. Multiply significands
 - $1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^5$
- ❑ 3. Normalize result & check for over/underflow
 - 1.0212×10^6
- ❑ 4. Round and renormalize if necessary
 - 1.021×10^6
- ❑ 5. Determine sign of result from signs of operands
 - $+1.021 \times 10^6$

Floating-Point Multiplication

- ❑ Now consider a 4-digit binary example
 - $1.000_2 \times 2^{-1} \times -1.110_2 \times 2^{-2}$ ($= 0.5 \times -0.4375$)
- ❑ 1. Add exponents
 - Unbiased: $-1 + -2 = -3$
 - Biased: $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127$
- ❑ 2. Multiply significands
 - $1.000_2 \times 1.110_2 = 1.110_2 \Rightarrow 1.110_2 \times 2^{-3}$
- ❑ 3. Normalize result & check for over/underflow
 - $1.110_2 \times 2^{-3}$ (no change) with no over/underflow
- ❑ 4. Round and renormalize if necessary
 - $1.110_2 \times 2^{-3}$ (no change)
- ❑ 5. Determine sign: $+ve \times -ve \Rightarrow -ve$
 - $-1.110_2 \times 2^{-3} = -0.21875$

FP Arithmetic Hardware

- ❑ FP multiplier is of similar complexity to FP adder
 - But uses a multiplier for significands instead of an adder
- ❑ FP arithmetic hardware usually does
 - Addition, subtraction, multiplication, division, reciprocal(倒數), square-root
 - $FP \leftrightarrow$ integer conversion (轉換)
- ❑ Operations usually takes several cycles
 - Can be pipelined

FP Instructions in MIPS

- ❑ FP hardware is coprocessor 1
 - Adjunct processor that extends the ISA
- ❑ Separate FP registers
 - 32 single-precision: \$f0, \$f1, ... \$f31
 - Paired for double-precision: \$f0/\$f1, \$f2/\$f3, ...
 - Release 2 of MIPS ISA supports 32×64 -bit FP reg's
- ❑ FP instructions operate only on FP registers
 - Programs generally don't do integer ops on FP data, or vice versa
(程序通常不會對 FP 資料執行整數操作，反之亦然)
 - More registers with minimal code-size impact
- ❑ FP load and store instructions
 - lwc1, ldc1, swc1, sdc1
 - e.g., ldc1 \$f8, 32(\$sp)

FP Instructions in MIPS

- ❑ Single-precision arithmetic
 - add.s, sub.s, mul.s, div.s
 - e.g., add.s \$f0, \$f1, \$f6
- ❑ Double-precision arithmetic
 - add.d, sub.d, mul.d, div.d
 - e.g., mul.d \$f4, \$f4, \$f6
- ❑ Single- and double-precision comparison
 - c.xx.s, c.xx.d (xx is eq, lt, le, ...)
 - Sets or clears FP condition-code bit
 - e.g. c.lt.s \$f3, \$f4
- ❑ Branch on FP condition code true or false
 - bc1t, bc1f
 - e.g., bc1t TargetLabel

FP Example: °F to °C

❑ C code:

- fahr in \$f12, result in \$f0, literals in global memory space

```
float f2c (float fahr) {  
    return ((5.0/9.0)*(fahr - 32.0));  
}
```

❑ Compiled MIPS code:

```
f2c: lwc1 $f16, const5($gp)  
     lwc2 $f18, const9($gp)  
     div.s $f16, $f16, $f18  
     lwc1 $f18, const32($gp)  
     sub.s $f18, $f12, $f18  
     mul.s $f0, $f16, $f18  
     jr $ra
```

FP Example: Array Multiplication

□ $X = X + Y \times Z$

- All 32×32 matrices, 64-bit double-precision elements

□ C code

- Addresses of x, y, z in \$a0, \$a1, \$a2, and i, j, k in \$s0, \$s1, \$s2

```
void mm (double x[][32], double y[][32], double z[][32]) {  
    int i, j, k;  
    for (i = 0; i != 32; i = i + 1)  
        for (j = 0; j != 32; j = j + 1)  
            for (k = 0; k != 32; k = k + 1)  
                x[i][j] = x[i][j] + y[i][k] * z[k][j];  
}
```

FP Example: Array Multiplication

□ MIPS code:

```
li $t1, 32      # $t1 = 32 (row size/loop end)
li $s0, 0       # i = 0; initialize 1st for loop
L1: li $s1, 0    # j = 0; restart 2nd for loop
L2: li $s2, 0    # k = 0; restart 3rd for loop
sll $t2, $s0, 5  # $t2 = i * 32 (size of row of x)
addu $t2, $t2, $s1 # $t2 = i * size(row) + j
sll $t2, $t2, 3  # $t2 = byte offset of [i][j]
addu $t2, $a0, $t2 # $t2 = byte address of x[i][j]
l.d $f4, 0($t2)  # $f4 = 8 bytes of x[i][j]
L3: sll $t0, $s2, 5 # $t0 = k * 32 (size of row of z)
addu $t0, $t0, $s1 # $t0 = k * size(row) + j
sll $t0, $t0, 3  # $t0 = byte offset of [k][j]
addu $t0, $a2, $t0 # $t0 = byte address of z[k][j]
l.d $f16, 0($t0) # $f16 = 8 bytes of z[k][j]
```


FP Example: Array Multiplication

```
...
sll $t0, $s0, 5      # $t0 = i*32 (size of row of y)
addu $t0, $t0, $s2    # $t0 = i*size(row) + k
sll $t0, $t0, 3      # $t0 = byte offset of [i][k]
addu $t0, $a1, $t0    # $t0 = byte address of y[i][k]
l.d $f18, 0($t0)     # $f18 = 8 bytes of y[i][k]
mul.d $f16, $f18, $f16 # $f16 = y[i][k] * z[k][j]
add.d $f4, $f4, $f16  # f4=x[i][j] + y[i][k]*z[k][j]
addiu $s2, $s2, 1     # $k k + 1
bne $s2, $t1, L3      # if (k != 32) go to L3
s.d $f4, 0($t2)       # x[i][j] = $f4
addiu $s1, $s1, 1     # $j = j + 1
bne $s1, $t1, L2      # if (j != 32) go to L2
addiu $s0, $s0, 1     # $i = i + 1
bne $s0, $t1, L1      # if (i != 32) go to L1
```

Accurate Arithmetic

□ units in the last place (ulp) 末位單位

- ulp 是「有效位元的最後位元」的單位量

- ulp可作為→量測誤差的單位

 - 誤差: 真實數字與可表達數字(有效位數)的差異

 - 以ulp 數量表達誤差量測數字

- IEEE Std 754保證誤差量為 $1/2$ ulp (1234捨、6789入、5一半機率捨或入)

 - $2.3400+0.0256=2.3656$ (真實計算數值)

 - 若可表達有效數字為小數2位，小數第2位是ulp

 - 使用GR，則有效數值為 2.37

 - 後兩位若00~49則捨、若51~99則入。

 - 後兩位是 guard bit (G), round bit (R)

 - 不使用GR，則有效數值為 2.36 (全捨去)

Accurate Arithmetic

IEEE Std 754 specifies additional rounding control

Choice of rounding modes (4種)

- Round to positive infinity (正數無條件入)
- Round to negative infinity (正數無條件捨)
- Round to zero

Mode	Example Value			
	+11.5	+12.5	-11.5	-12.5
to $+\infty$	+12.0	+13.0	-11.0	-12.0
to $-\infty$	+11.0	+12.0	-12.0	-13.0
to 0	+11.0	+12.0	-11.0	-12.0
to even	+12.0	+12.0	-12.0	-12.0

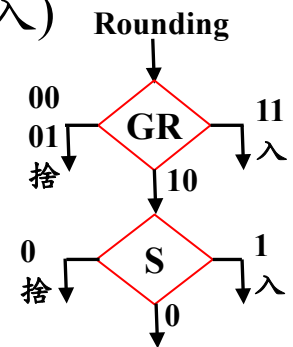
- unbiased to the nearest even (4捨6入5往有效位之偶數入)

significand 有效位			extra bit (額外位)	
		LSB	G	R
			0	0
			0	1
		0	1	0
		1	1	0

捨
捨
捨
入

(G,R)=(1, 0) 代表有LSB的 $1/2=0.5$ ，LSB=1表示偶數

- Allows programmer to fine-tune numerical behavior of a computation



Accurate Arithmetic

□ IEEE Std 754 specifies additional rounding control

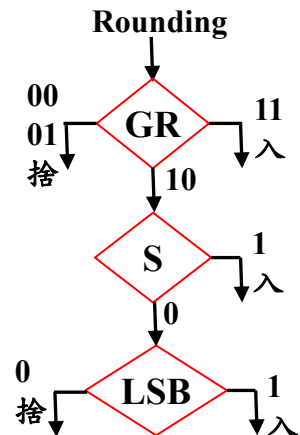
○ Extra bits of precision (**G**uard, **R**ound, **S**ticky) (第5種)

➤ (guard bit, round bit)

– (0, 0), (0, 1) 捨去

– (1, 1) 進位

– (1, 0), sticky bit=1 進位, sticky bit=0 捨去



□ Not all FP units implement all options

○ Most programming languages and FP libraries just use defaults

□ Trade-off between hardware complexity, performance, and market requirements

Exercise

□ 浮點數加法運算，運算完mantissa(小數部位)32 bit (rounding)

○ IEEE 單精度mantissa有23 bit

○ G是mantissa最後一個bit的右邊第一位、R是G右邊一個bit、S是R右邊所有bit做OR運算

1.1111 0000 1111 0000 1101 **101**1 0011 1100 $\times 2^{15}$ (normalized)

23 bit↑

○ Guard bit是第24bit、Round bit是第25bit

1.1111 0000 1111 0000 1101 1011 0**0**11 1100

GRS-----or

○ GR=10, S=1 入

○ 答案

1.1111 0000 1111 0000 1101 **110** $\times 2^{15}$ (normalized)

23 bit↑

Interpretation of Data

□ The BIG Picture

- Bits have no inherent meaning
 - Interpretation depends on the instructions applied
 - (每個位元沒有既定的意義，完全依賴指令的定義與解釋)
- Computer representations of numbers
 - Finite range and precision
 - Need to account for this in programs
 - (程式設計應考慮浮點數字有限範圍與其精確度運算)

Associativity

- ❑ Parallel programs may interleave operations in unexpected orders
 - Assumptions of associativity may fail (因為浮點數精確度與表示式，結合性可能失效)

		$(x+y)+z$	$x+(y+z)$
x	-1.50E+38	0.00E+00	-1.50E+38
y	1.50E+38		1.50E+38
z	1.0		
		1.00E+00	0.00E+00

- Need to validate parallel programs under varying degrees of parallelism

Right Shift and Division

- Left shift by i places multiplies an integer by 2^i
 - 乘法 $2^i \Rightarrow$ 左移 i 位
- Right shift divides by 2^i
 - Only for unsigned integers
 - 除法 $2^i \Rightarrow$ 右移 i 位，只適用於無號整數
- signed integers 有號整數
 - Arithmetic right shift: replicate the sign bit
 - e.g., $-5 / 4 = -1.25$
 - $11111011_2 \gg 2 = 1111110_2 = -2$
 - Rounds toward $-\infty$
- 無號數 $11111011_2 \gg 2 = 00111110_2 = +62$

Concluding Remarks

- ❑ ISAs support arithmetic
 - Signed and unsigned integers
 - Floating-point approximation to reals
- ❑ Bounded range and precision
 - Operations can overflow and underflow
- ❑ MIPS ISA
 - Core instructions: 54 most frequently used
 - 100% of SPECINT, 97% of SPECFP
 - Other instructions: less frequent